RAPPORT TECHNIQUE DE PROJET

Déploiement et sécurisation d'un serveur Linux sous Ubuntu

Administration système et cybersécurité

Réalisé par

MBWINI MUGBAKRANDE Geni

Contexte

Projet personnel

Environnement

Ubuntu Server 26.04.1 LTS • Oracle VirtualBox • Ubuntu Desktop 24.04 LTS (hôte)

Octobre 2026

Table des matières

1. Introduction et cadre du projet **3**
1.1 Présentation générale 3
1.2 Contexte 3
1.3 Problématique 3
1.4 Objectifs 4
1.5 Périmètre, hypothèses et contraintes 4
1.6 Démarche méthodologique 5
1.7 Livrables 5
1.8 Organisation du document 5
2. Architecture, choix techniques et approche de sécurité **6**
2.1 Matériel et logiciels 6
2.2 Architecture réseau 6
2.3 Justification des choix techniques 6
2.4 Menaces et mesures associées 7
2.5 Feuille de route réalisée 7
3. Déploiement de la machine virtuelle et du réseau **8**
3.1 Création de la machine virtuelle 8
3.2 Vérification du système et mises à jour 9
3.3 Installation d'OpenSSH Server 10
3.4 Configuration du réseau Host-Only 12
3.5 Vérification des adresses et de la connectivité 14
3.6 Première connexion SSH et configuration initiale 15
4. Sécurisation du serveur **17**
4.1 Utilisateurs, groupes et permissions 17
4.2 Pare-feu UFW 18
4.3 Contrôle des ports exposés 19
4.4 Authentification par clés SSH 20
4.5 Durcissement de SSH 22
4.6 Protection contre les tentatives répétées : Fail2ban 24
5. Journaux, surveillance et sauvegarde **28**
5.1 Analyse des journaux 28
5.2 Surveillance des ressources 28
5.3 Sauvegarde et restauration 29
6. Audit de sécurité avec Lynis **32**
7. Problèmes rencontrés et solutions **35**
8. Bilan avant / après **36**
8.1 Limites constatées 36
8.2 Pistes d'amélioration 36
9. Conclusion **37**
9.1 Synthèse 37
9.2 Réponse à la problématique 37
9.3 Compétences acquises 37
9.4 Perspectives 37

1. Introduction et cadre du projet

1.1 Présentation générale

Le projet « Déploiement et sécurisation d'un serveur Linux sous Ubuntu » consiste à installer, configurer, sécuriser et documenter un serveur Ubuntu Server dans un environnement virtualisé. Il s'agit d'un projet personnel d'administration système et de cybersécurité, mené comme un laboratoire pratique : chaque mesure de sécurité est appliquée, puis vérifiée par un test concret et illustrée par une capture d'écran.

Le serveur est hébergé dans une machine virtuelle Oracle VirtualBox, exécutée sur un ordinateur portable sous Ubuntu Desktop 24.04 LTS. Il est administré exclusivement à distance, par SSH, depuis l'ordinateur hôte, à travers un réseau privé isolé. Cette organisation reproduit, à petite échelle, la situation d'un administrateur qui gère un serveur distant sans y accéder physiquement.

Le résultat attendu est double : disposer d'un serveur fonctionnel et durci, et conserver une documentation technique professionnelle, réutilisable dans un portfolio, qui démontre concrètement les compétences acquises.

1.2 Contexte

L'administration d'un serveur Linux est une compétence centrale en informatique : la majorité des services en ligne (sites web, bases de données, messageries, outils internes) reposent sur des serveurs Linux accessibles à distance. Un serveur raccordé à un réseau est, par nature, exposé : dès qu'un service d'accès à distance comme SSH est ouvert, il peut faire l'objet de tentatives de connexion automatisées, de scans de ports et d'essais de mots de passe.

Dans ce contexte, installer simplement un système et ses services ne suffit pas. Un serveur correctement administré doit être mis à jour, limiter les droits de chacun, ne laisser ouvert que ce qui est nécessaire, résister aux attaques par force brute, conserver des traces exploitables et pouvoir être restauré après un incident. Ces exigences constituent le socle de ce que l'on appelle le durcissement (hardening) d'un système.

Le choix d'un laboratoire virtualisé répond à plusieurs contraintes pratiques. Une installation directe sur le matériel physique avait posé des difficultés sur un autre ordinateur ; la machine virtuelle a donc été retenue. Elle permet aussi d'expérimenter sans risque pour le système de l'hôte, de conserver une console de secours indépendante du réseau (la console VirtualBox) et de reproduire facilement l'environnement.

1.3 Problématique

Comment déployer un serveur Ubuntu administrable à distance, puis réduire sa surface d'attaque et le rendre résistant aux tentatives d'intrusion courantes, sans jamais perdre l'accès d'administration et en prouvant l'efficacité de chaque mesure ?

Cette question générale se décline en plusieurs questions plus précises auxquelles le projet apporte des réponses vérifiées :

- Comment permettre l'administration à distance depuis l'hôte sans exposer le serveur à Internet ?
- Comment organiser les comptes et les permissions selon le principe du moindre privilège ?
- Comment limiter les connexions entrantes aux seuls flux nécessaires, et le démontrer ?

- Comment sécuriser SSH au-delà de la configuration par défaut, sans se verrouiller soi-même ?
- Comment détecter et bloquer automatiquement les tentatives répétées de connexion ?
- Comment s'assurer que les données et les configurations peuvent être restaurées en cas de perte ?
- Comment mesurer objectivement le niveau de sécurité obtenu ?

La difficulté principale tient à l'équilibre entre sécurité et accessibilité : un pare-feu ou une configuration SSH trop restrictive, appliqués dans le mauvais ordre, peuvent couper l'accès au serveur. La démarche suivie vise précisément à éviter ce piège (voir 1.6).

1.4 Objectifs

Objectif général : construire un serveur Linux fonctionnel, sécurisé et documenté, et démontrer par des tests la réalité des protections mises en place.

Les objectifs spécifiques, tous atteints, sont les suivants :

N°	Objectif spécifique	Critère de réussite vérifié
O1	Déployer un serveur Ubuntu dans une machine virtuelle et le tenir à jour	Système installé, mises à jour appliquées, accès Internet validé
O2	Mettre en place une administration à distance par SSH via un réseau privé	Ping 0 % de perte, connexion SSH depuis l'hôte
O3	Gérer utilisateurs, groupes et permissions (moindre privilège)	Compte analyste sans sudo, répertoire partagé en 2770
O4	Configurer un pare-feu restrictif	UFW actif, seul le port 22 ouvert, scan Nmap concluant
O5	Durcir SSH et passer à l'authentification par clés	Mots de passe refusés, root interdit, accès par groupe
O6	Se protéger des tentatives répétées de connexion	Fail2ban : adresse bannie après 3 échecs
O7	Analyser les journaux et surveiller les ressources	Traçage de connexions, bannissements, charge et mémoire
O8	Sauvegarder et restaurer	Archive créée, restauration testée, copie hors serveur
O9	Auditer et documenter	Audit Lynis (indice 66 puis 67) et présent rapport

1.5 Périmètre, hypothèses et contraintes

Périmètre. Le projet couvre un serveur unique, administré par une seule personne, dans un réseau de laboratoire isolé. Il traite l'accès distant, les comptes, le pare-feu, la détection des intrusions par force brute, la journalisation, la surveillance de base, la sauvegarde et l'audit.

Hors périmètre. Le projet ne traite pas l'hébergement de services applicatifs (serveur web, base de données), la haute disponibilité, le chiffrement des disques, ni la supervision centralisée. Il ne vise pas non plus à exposer le serveur à Internet.

Hypothèses. Le réseau Host-Only est considéré comme fiable et accessible uniquement depuis l'hôte ; l'ordinateur hôte est lui-même sous le contrôle de l'administrateur ; le réseau NAT sert uniquement aux mises à jour.

Contraintes. Ressources limitées (environ 8 Go de mémoire sur l'hôte, dont 4 Go pour la VM), nécessité de ne jamais perdre l'accès d'administration, et obligation de conserver l'installation existante (Ubuntu Server 26.04.1 LTS) sans réinstallation inutile.

1.6 Démarche méthodologique

Le projet a été conduit étape par étape, selon une règle simple : observer avant de modifier, modifier de façon réversible, puis vérifier par un test. Cette démarche s'appuie sur quatre principes :

- **Moindre privilège :** chaque compte ne dispose que des droits dont il a besoin.
- **Défense en profondeur :** plusieurs couches indépendantes (pare-feu, SSH durci, Fail2ban, droits sur les fichiers) se complètent.
- **Réduction de la surface d'attaque :** un seul port exposé, seulement depuis le réseau d'administration, et uniquement par clés.
- **Traçabilité et preuve :** chaque mesure est accompagnée de la commande, du résultat observé et d'une capture.

L'ordre des étapes est lui-même une mesure de sécurité. Le réseau et la connexion SSH depuis l'hôte ont été validés avant l'activation du pare-feu ; les règles UFW ont été préparées avant d'être activées ; les clés SSH ont été testées pour les deux comptes avant de désactiver les mots de passe ; la syntaxe de SSH a été contrôlée (sshd -t) avant tout rechargement ; et une session d'administration ainsi que la console VirtualBox sont restées ouvertes pendant les changements sensibles.

1.7 Livrables

- Un serveur Ubuntu configuré et durci, en fonctionnement dans VirtualBox.
- Une archive de sauvegarde datée, restaurée avec succès, et sa copie sur l'hôte.
- Un jeu de 57 captures d'écran numérotées, illustrant chaque étape.
- Le présent rapport technique.

1.8 Organisation du document

Chapitre	Contenu
2	Architecture matérielle, logicielle et réseau
3	Déploiement de la machine virtuelle, d'OpenSSH et du réseau Host-Only
4	Sécurisation : comptes, pare-feu, ports, clés SSH, durcissement SSH, Fail2ban
5	Journaux, surveillance des ressources, sauvegarde et restauration
6	Audit de sécurité avec Lynis
7	Problèmes rencontrés et solutions
8	Bilan avant / après, limites et pistes d'amélioration
9	Conclusion

2. Architecture, choix techniques et approche de sécurité

Ce chapitre décrit l'environnement du laboratoire, explique pourquoi chaque composant a été retenu et précise les risques que les mesures de sécurité cherchent à couvrir.

2.1 Matériel et logiciels

Élément	Configuration
Ordinateur hôte	HP ProBook 640 G2, environ 8 Go de mémoire vive
Système hôte	Ubuntu Desktop 24.04 LTS
Virtualisation	Oracle VirtualBox
Système invité	Ubuntu Server 26.04.1 LTS, noyau 7.0.0-38-generic, x86-64
Ressources de la VM	4 Go de mémoire vive, 2 processeurs virtuels, disque virtuel de 25 Go
Nom d'hôte du serveur	UbuntuServer
Compte administrateur	genius_24 (membre du groupe sudo)
Compte standard	analyste (sans droits sudo)

2.2 Architecture réseau

La machine virtuelle dispose de deux adaptateurs réseau. Le premier, en mode NAT, donne accès à Internet pour les mises à jour. Le second, en mode réseau privé hôte (Host-Only), permet l'administration du serveur depuis l'ordinateur hôte sans l'exposer à un réseau extérieur.

Adaptateur	Mode	Interface / adresse	Rôle
Adaptateur 1	NAT	enp0s3 – 10.0.2.15/24	Accès Internet depuis la VM
Adaptateur 2	Host-Only	enp0s8 – 192.168.56.101/24	Administration depuis l'hôte
Hôte	Host-Only	vboxnet0 – 192.168.56.1/24	Poste d'administration

2.3 Justification des choix techniques

Choix	Justification
Machine virtuelle	Permet d'expérimenter sans risque pour le poste hôte, d'arrêter et de redémarrer le serveur à volonté, et de disposer d'une console de secours indépendante du réseau.
Ubuntu Server LTS	Distribution très répandue en entreprise, dotée d'un support à long terme et d'une documentation abondante, avec des outils de sécurité intégrés (UFW, journald).
Deux adaptateurs réseau	Sépare l'accès Internet (NAT, sortant uniquement) de l'administration (Host-Only, réseau privé). Le serveur n'est joignable que depuis l'hôte.
SSH par clés	Une clé Ed25519 est bien plus difficile à compromettre qu'un mot de passe et supprime le risque d'attaque par force brute sur les mots de passe.

Choix	Justification
UFW	Interface simple au pare-feu du noyau, adaptée à une politique « tout refuser sauf le nécessaire ». Sa syntaxe lisible réduit le risque d'erreur.
Fail2ban	Complète le pare-feu en réagissant automatiquement aux comportements suspects observés dans les journaux de SSH.
Lynis	Outil d'audit libre qui donne un indice de durcissement et des recommandations, utile pour mesurer objectivement le résultat.

2.4 Menaces et mesures associées

Le tableau suivant relie les risques envisagés aux mesures mises en œuvre dans ce projet.

Risque	Mesure	Vérification
Exposition de services inutiles	Pare-feu UFW en refus par défaut	Scan Nmap, ss -lntup
Force brute sur SSH	Clés uniquement, MaxAuthTries 3, Fail2ban	Refus sans clé, adresse bannie
Connexion directe en root	PermitRootLogin no	sshd -T
Compte compromis avec trop de droits	Compte standard sans sudo, groupes dédiés	sudo -l, droits 2770
Accès à un répertoire par un tiers	Droits 2770 sur /srv/projet	Test d'écriture, ls -l
Perte ou altération de données	Sauvegarde datée, copie hors serveur	Restauration comparée (diff)
Erreur de configuration	Test de syntaxe, session de secours	sshd -t, console VirtualBox

2.5 Feuille de route réalisée

- Installation et mise à jour du système, installation d'OpenSSH.
- Configuration du réseau Host-Only et validation de la connectivité.
- Utilisateurs, groupes, répertoire partagé et permissions.
- Pare-feu UFW et contrôle des ports.
- Authentification par clés puis durcissement de SSH.
- Fail2ban, journaux et surveillance.
- Sauvegarde, restauration et audit Lynis.

3. Déploiement de la machine virtuelle et du réseau

Ce chapitre décrit la mise en place de la base du laboratoire : la création de la machine virtuelle, l'installation et la mise à jour du système, le service SSH, puis le réseau qui permet d'administrer le serveur depuis l'hôte. L'objectif est de disposer d'un serveur fonctionnel et joignable avant d'engager la moindre mesure de sécurité, afin que chaque modification ultérieure puisse être testée.

3.1 Création de la machine virtuelle

La machine virtuelle a été créée dans VirtualBox à partir de l'image ISO d'Ubuntu Server 26.04.1 LTS. Les ressources attribuées sont de 4 Go de mémoire vive, 2 processeurs virtuels et un disque de 25 Go. Le mot de passe saisi dans l'assistant a été masqué sur la capture ci-dessous.

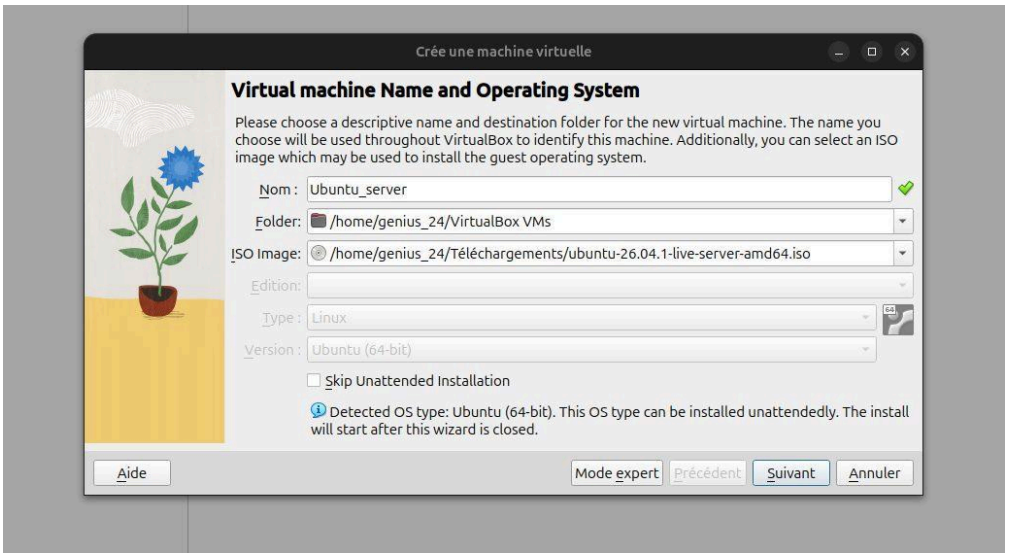


Figure 1 – Assistant VirtualBox : nom de la VM et image ISO

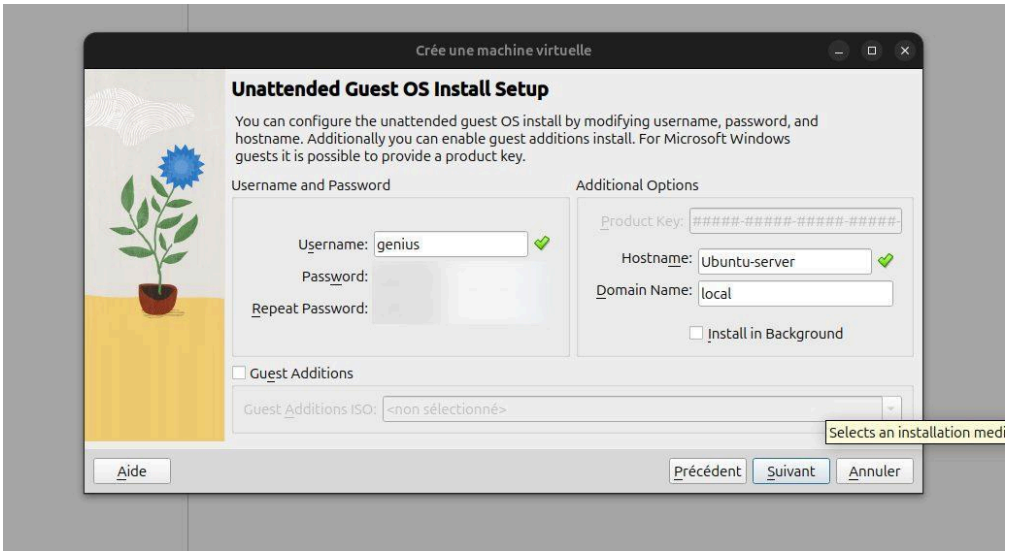


Figure 2 – Paramètres d'installation : nom d'hôte et compte (mot de passe masqué)

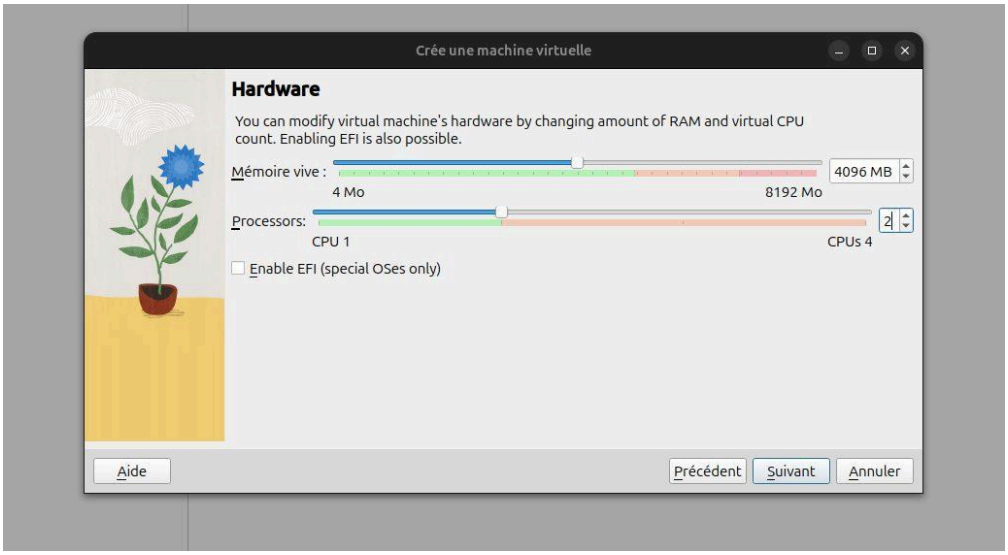


Figure 3 – Mémoire vive (4 096 Mo) et processeurs (2)

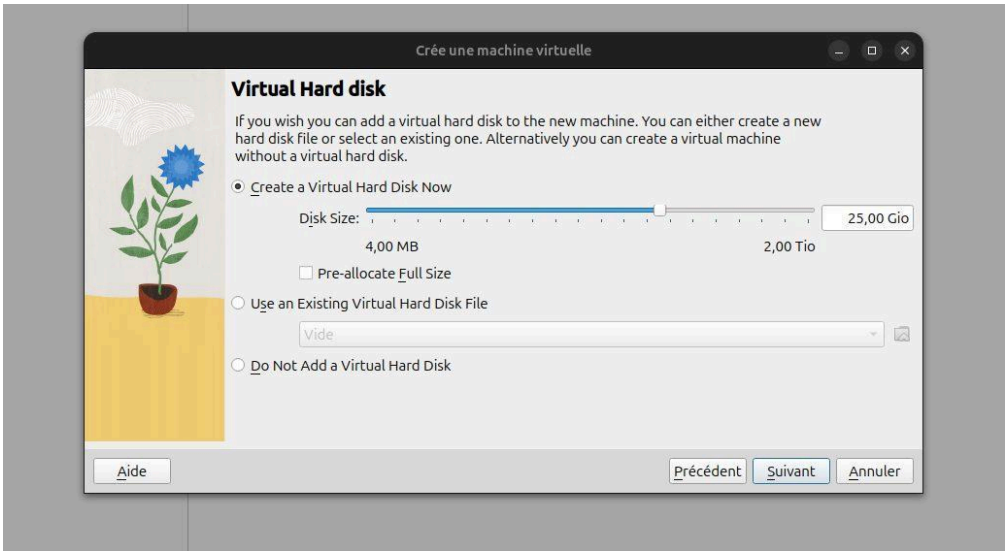


Figure 4 – Disque virtuel de 25 Go

3.2 Vérification du système et mises à jour

Après l'installation, le système a été vérifié avec hostnamectl, puis les mises à jour ont été appliquées. Un contrôle de la nécessité d'un redémarrage a également été effectué.

```
hostnamectl
ping -c 4 1.1.1.1
ping -c 4 ubuntu.com
sudo apt update
sudo apt upgrade
```

Les tests de connectivité ont montré quatre paquets transmis et reçus, sans perte, ce qui valide à la fois l'accès IP et la résolution DNS.

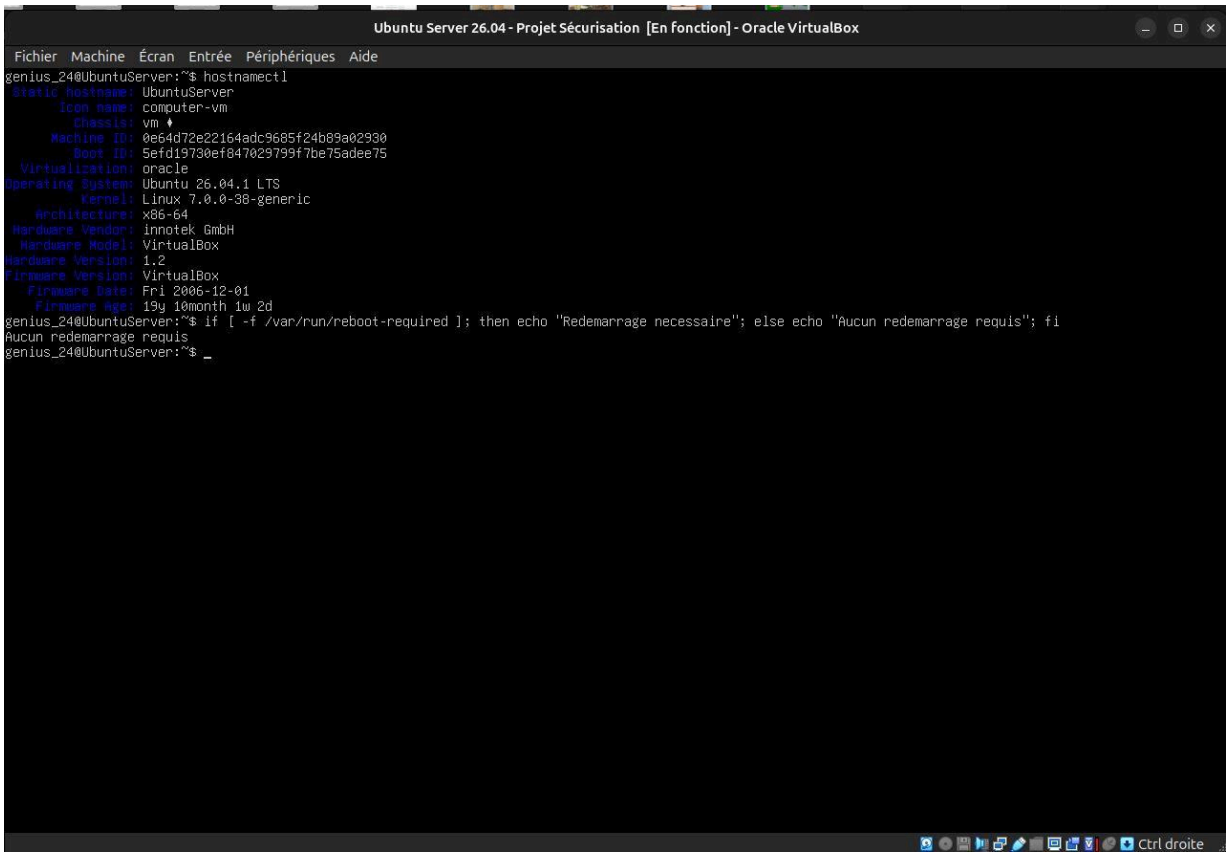


Figure 5 – Vérification du système et des mises à jour

3.3 Installation d'OpenSSH Server

Le service SSH n'étant pas présent, OpenSSH Server a été installé, activé au démarrage et démarré. Son état et l'écoute sur le port TCP 22 ont ensuite été contrôlés.

```
sudo apt install openssh-server
sudo systemctl enable --now ssh
systemctl status ssh --no-pager
sudo ss -lntp | grep ':22'
```

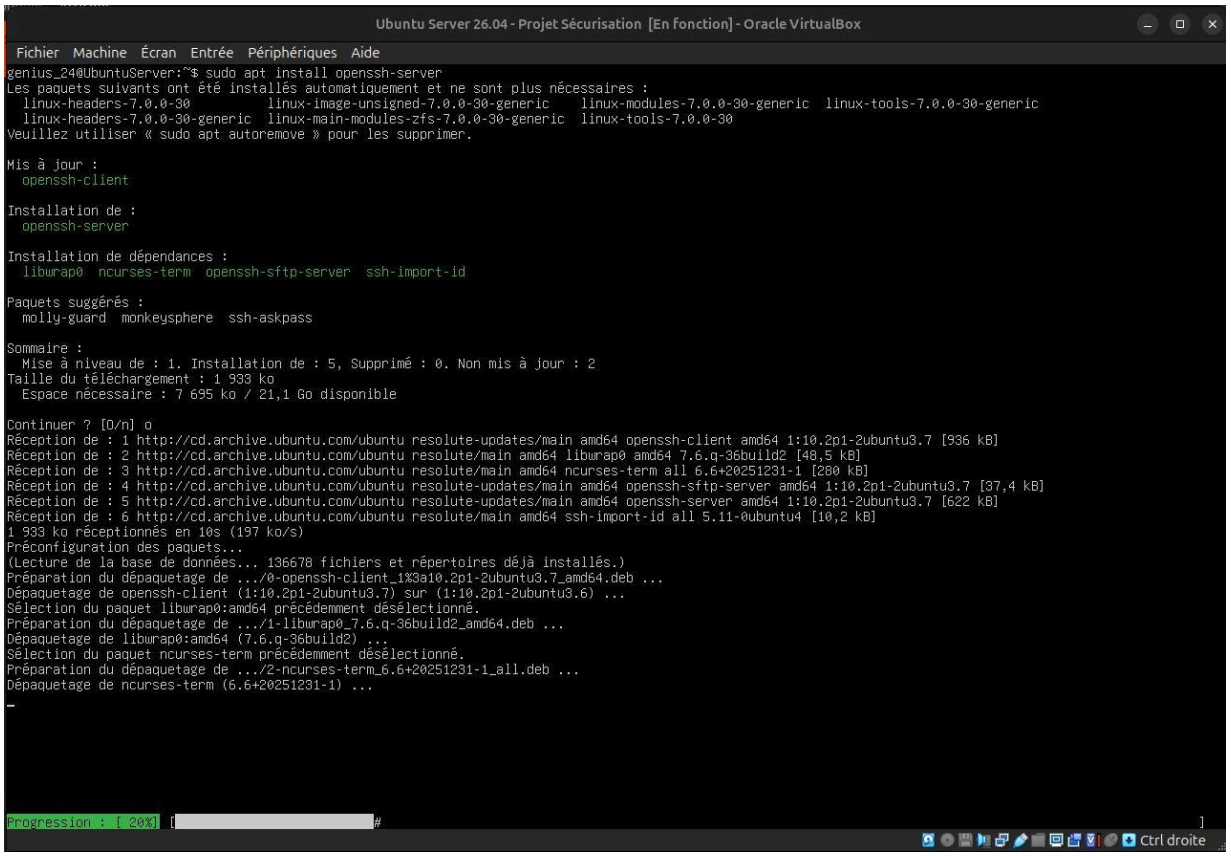


Figure 6 – Installation d'OpenSSH Server

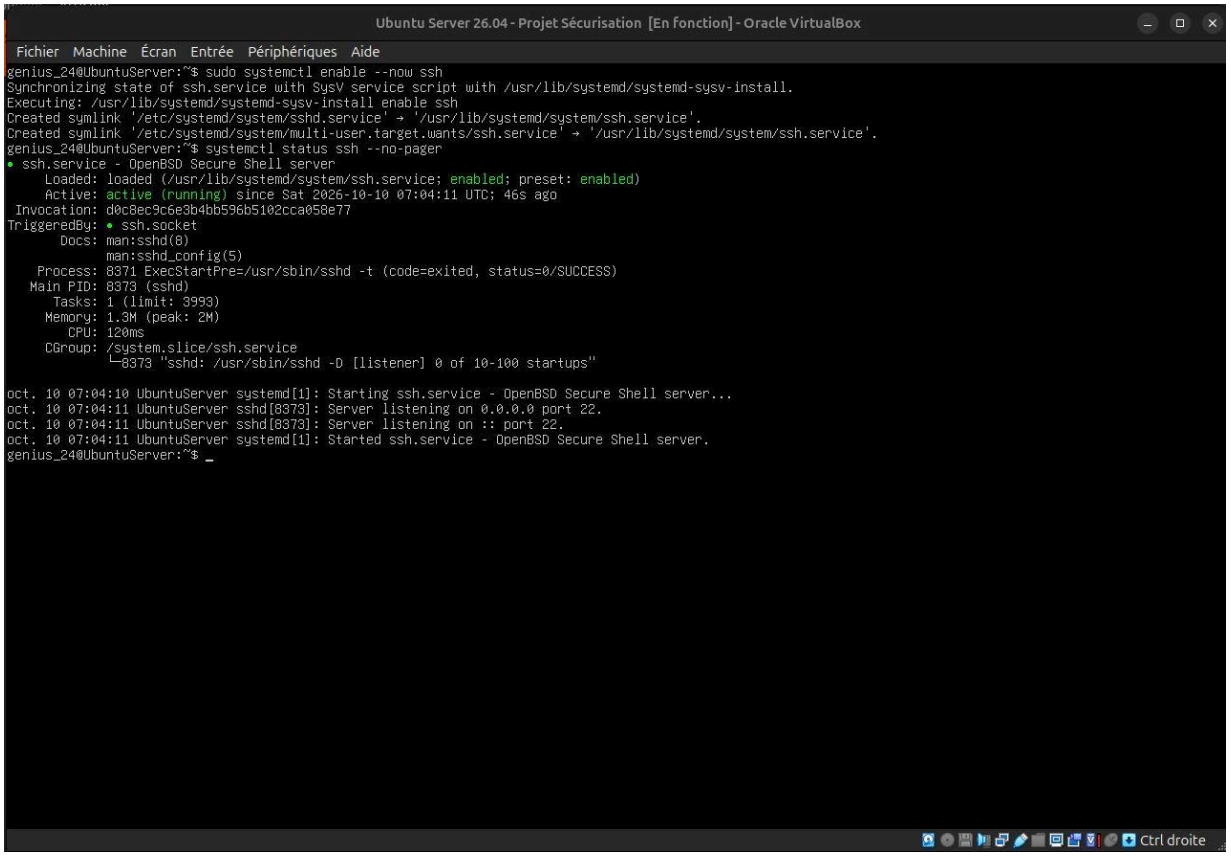


Figure 7 – Service SSH actif et démarrage automatique

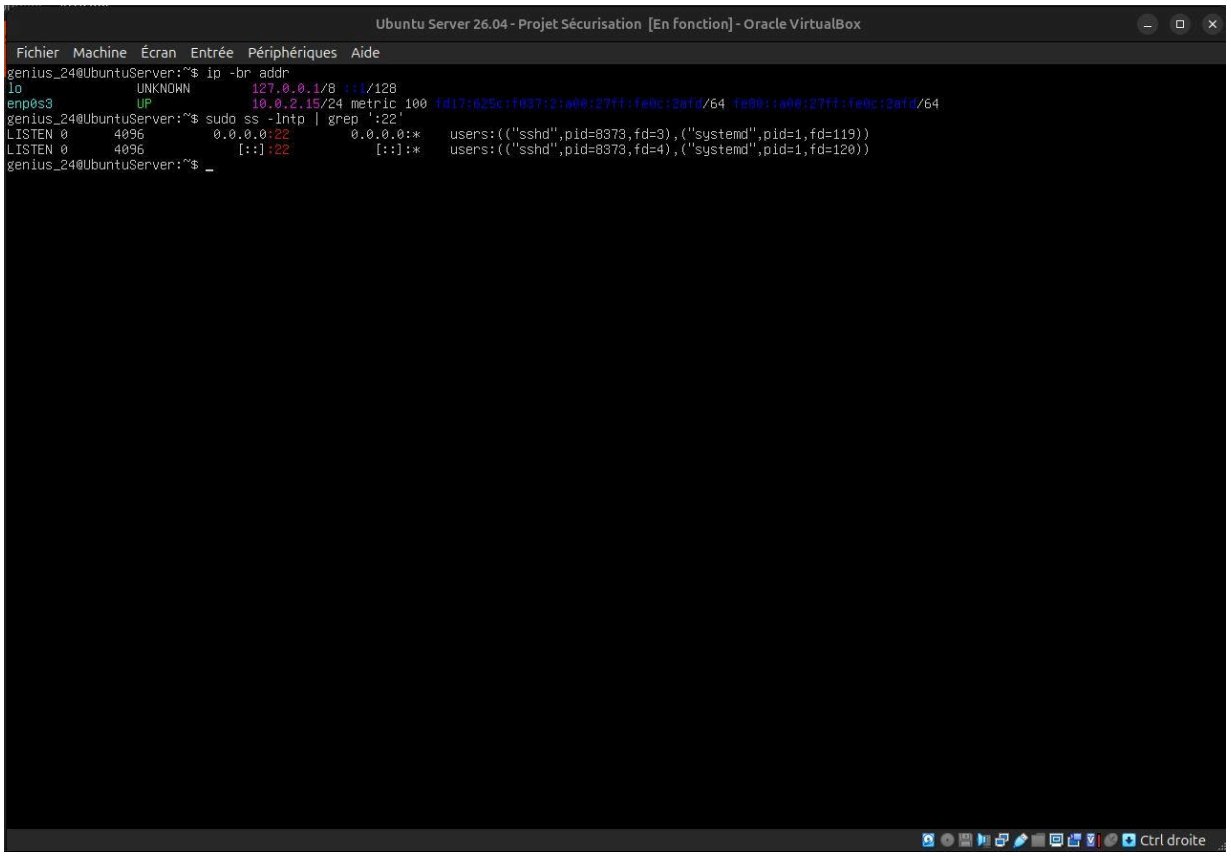


Figure 8 – Interfaces réseau et écoute sur le port 22

3.4 Configuration du réseau Host-Only

Aucune interface Host-Only n'existant dans VirtualBox, un réseau privé hôte (vboxnet0) a été créé, puis associé à l'adaptateur 2 de la machine virtuelle. L'adaptateur 1 est resté en mode NAT.

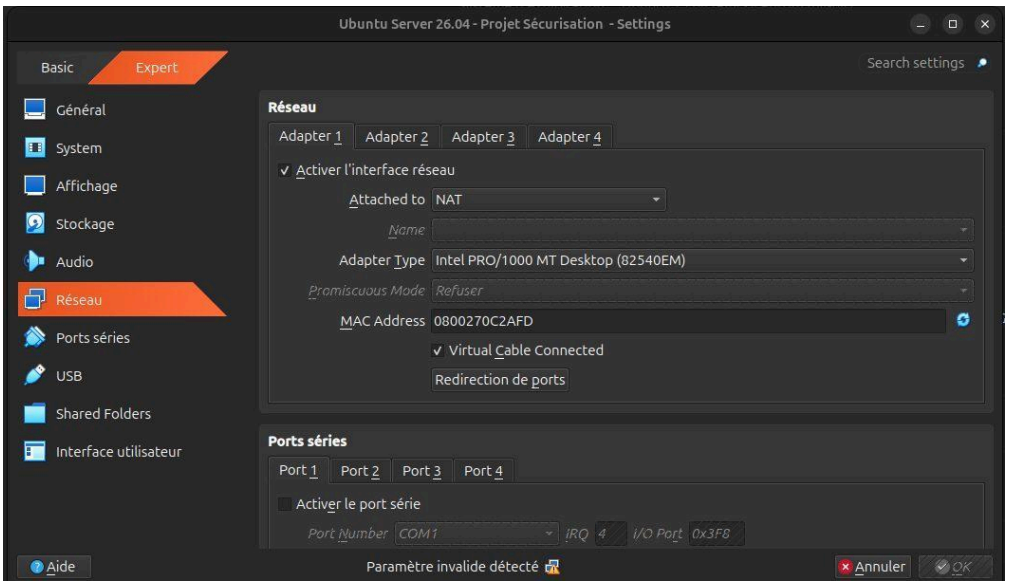


Figure 9 – Adaptateur 1 en mode NAT (avant modification)

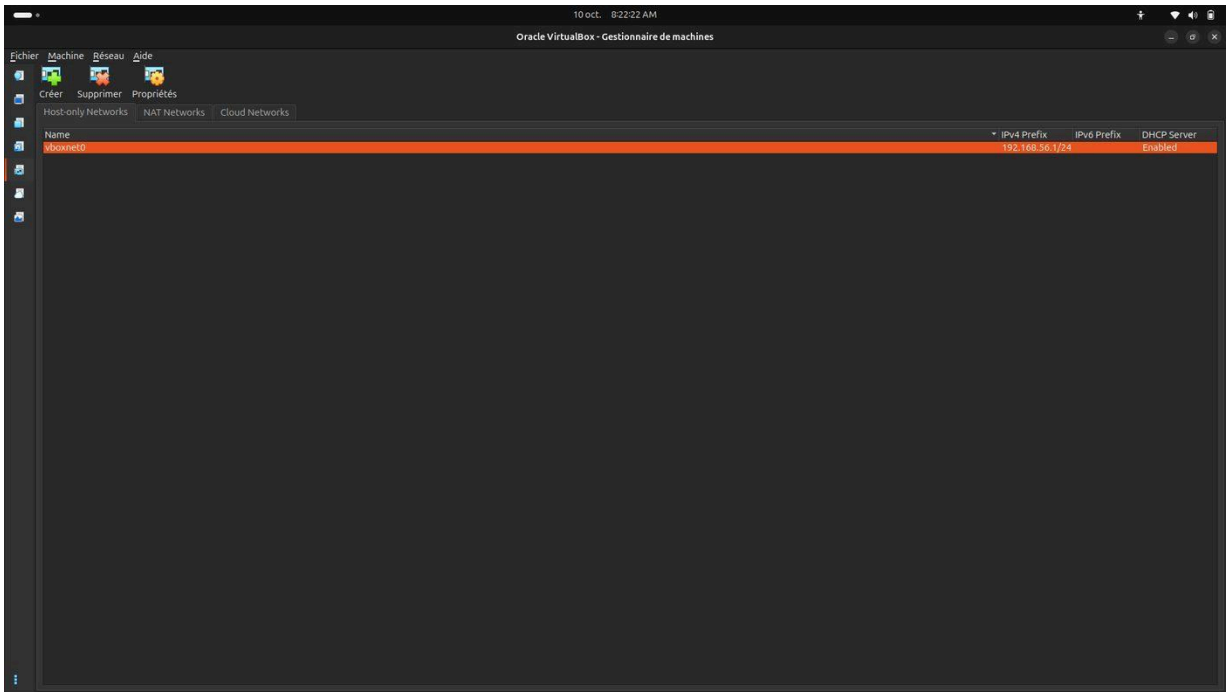


Figure 10 – Création du réseau Host-Only dans VirtualBox

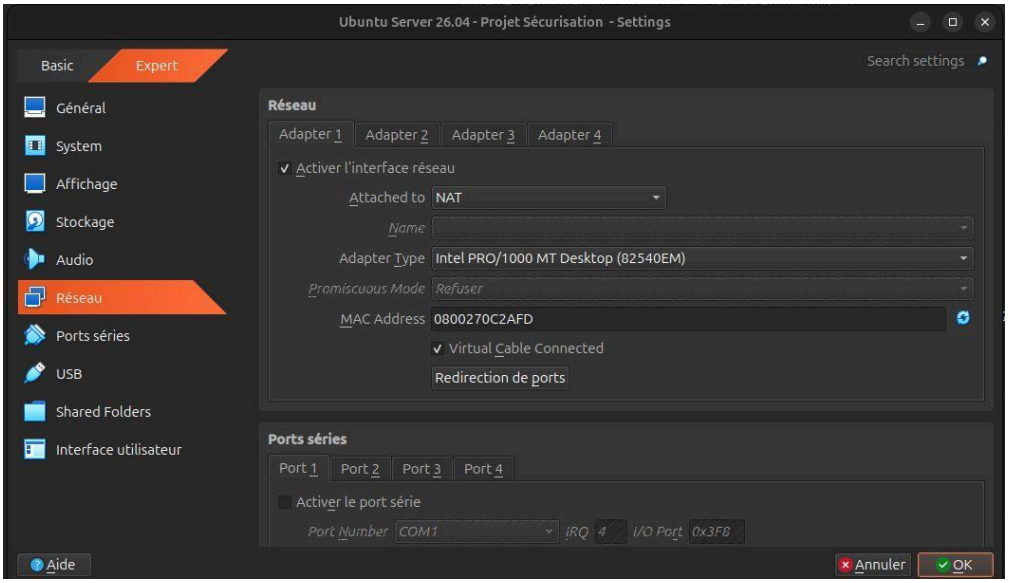


Figure 11 – Adaptateur 1 conservé en mode NAT

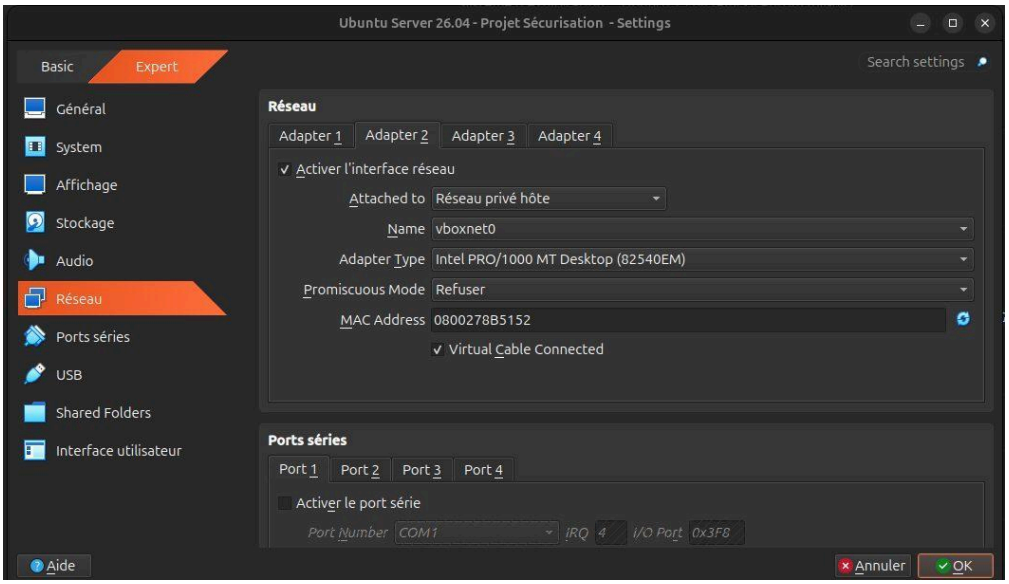


Figure 12 – Adaptateur 2 associé au réseau privé hôte vboxnet0

3.5 Vérification des adresses et de la connectivité

Sur le serveur, la commande ip -br addr confirme les deux interfaces : enp0s3 (NAT, 10.0.2.15/24) et enp0s8 (Host-Only, 192.168.56.101/24). Sur l'hôte, l'interface vboxnet0 porte l'adresse 192.168.56.1/24, dans le même réseau.

```
ip -br addr
```

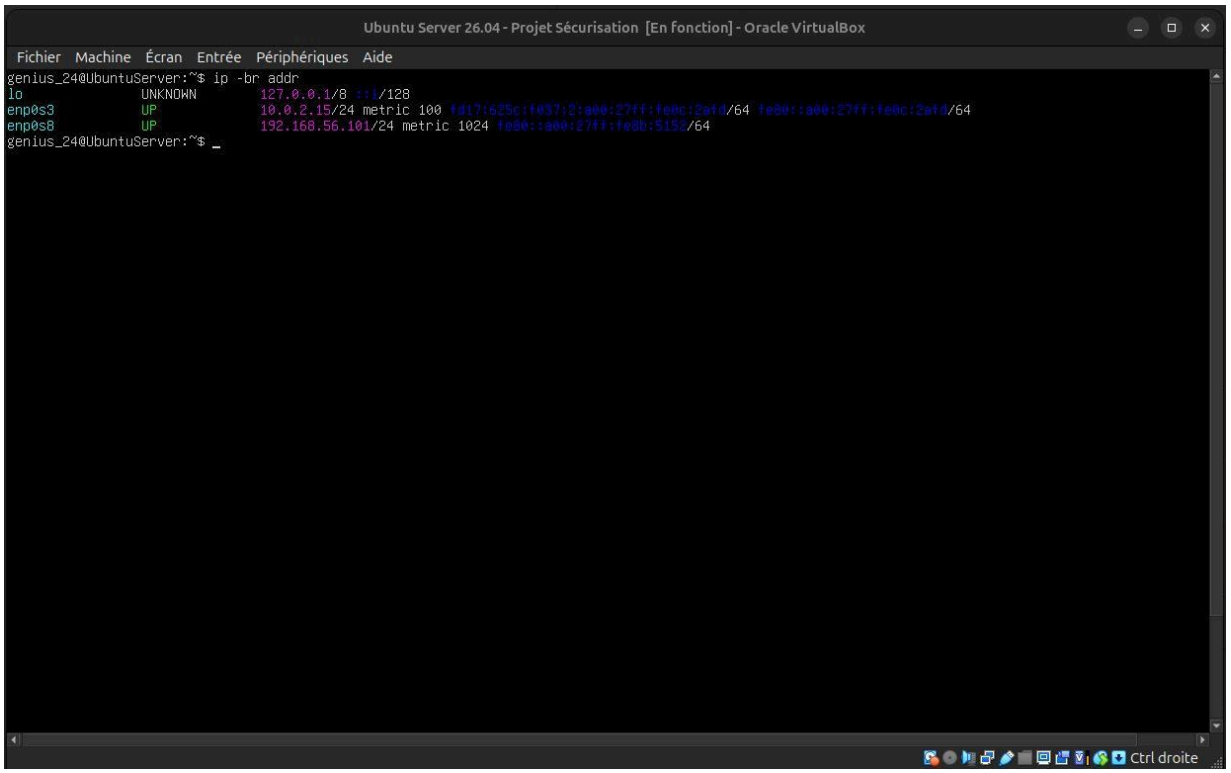


Figure 13 – Interfaces du serveur : NAT (enp0s3) et Host-Only (enp0s8)

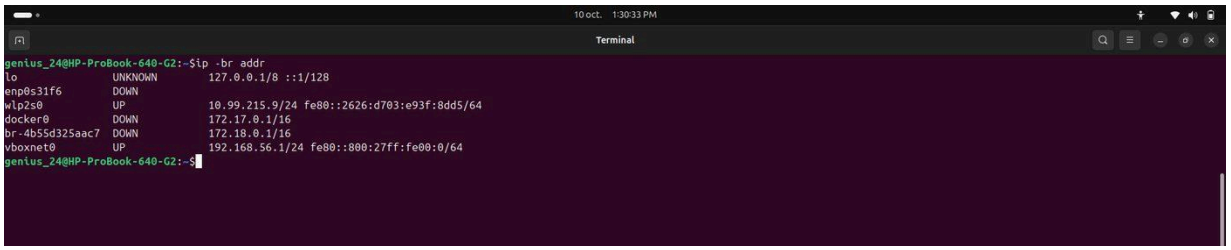


Figure 14 – Interface vboxnet0 de l'ordinateur hôte (192.168.56.1/24)

Le test de connectivité depuis l'hôte vers le serveur a abouti : 4 paquets transmis, 4 reçus, 0 % de perte, avec un temps moyen d'environ 0,85 ms.

```
ping -c 4 192.168.56.101
```

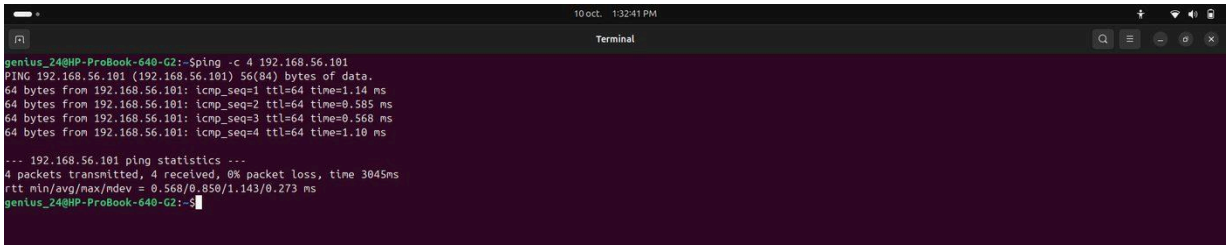


Figure 15 – Ping de l'hôte vers le serveur : 0 % de perte

3.6 Première connexion SSH et configuration initiale

La première connexion SSH depuis l'hôte a été établie avec le compte genius_24. L'empreinte du serveur a été acceptée, puis l'identité du serveur a été confirmée avec hostname et ip -br addr.

```
ssh genius_24@192.168.56.101
hostname
ip -br addr
```

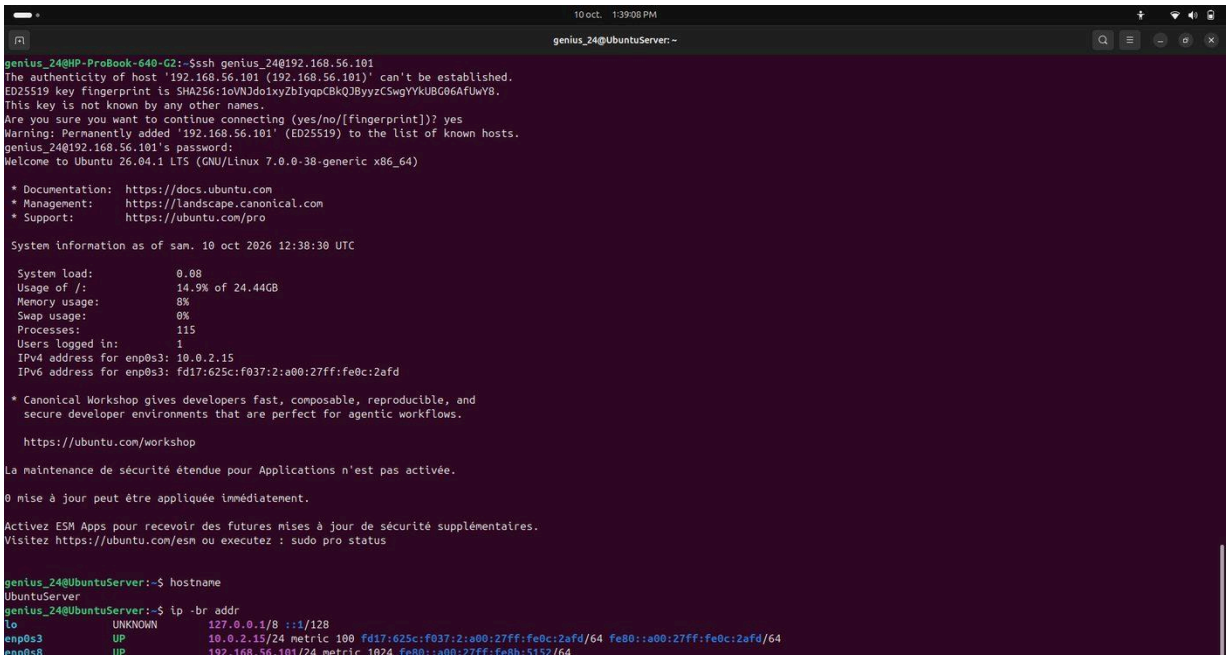


Figure 16 – Première connexion SSH depuis l'hôte

Avant tout durcissement, la configuration SSH effective a été relevée afin de servir d'état de référence.

```
sudo sshd -T | grep -E
'^(port|permitrootlogin|passwordauthentication|pubkeyauthentication|maxauthtries|x11forwar
ding|allowusers|allowgroups)'
```

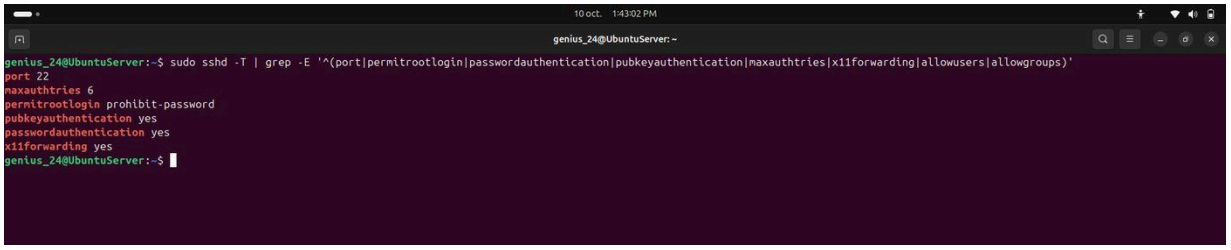


Figure 17 – Configuration SSH initiale (état de référence)

On relève le port 22, 6 tentatives d'authentification autorisées, la connexion root par mot de passe interdite, l'authentification par mot de passe et par clé acceptées, le transfert X11 actif et aucune restriction par utilisateur ou par groupe.

4. Sécurisation du serveur

Ce chapitre constitue le cœur du projet. Les mesures sont appliquées dans un ordre volontaire : on organise d'abord les comptes et les droits, on filtre ensuite le réseau, puis on remplace les mots de passe par des clés avant de durcir SSH, et on termine par la protection automatique contre les tentatives répétées. Cet ordre garantit qu'aucune mesure ne coupe l'accès d'administration avant que son remplaçant ait été validé.

4.1 Utilisateurs, groupes et permissions

Au départ, un seul compte humain existait : genius_24 (UID 1000), seul membre du groupe sudo. Conformément au principe du moindre privilège, un compte standard analyste a été créé, ainsi que deux groupes : sshusers (accès SSH, utilisé plus tard avec AllowGroups) et projet (accès au répertoire partagé).

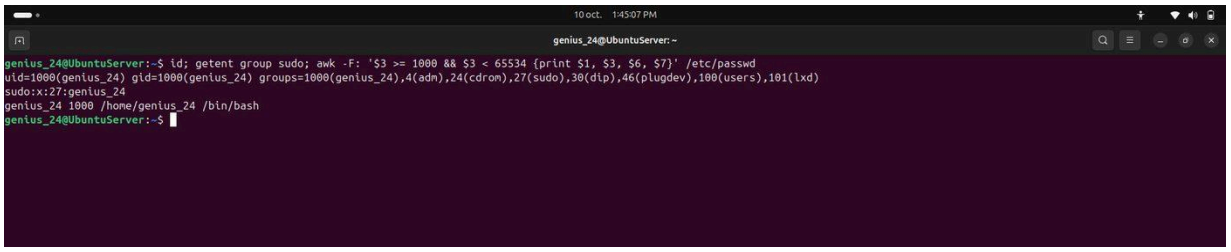


Figure 18 – État initial des comptes et des groupes

```
sudo groupadd sshusers
sudo groupadd projet
sudo adduser analyste
sudo usermod -aG sshusers,projet analyste
sudo usermod -aG sshusers,projet genius_24
id analyste
getent group sshusers projet
```

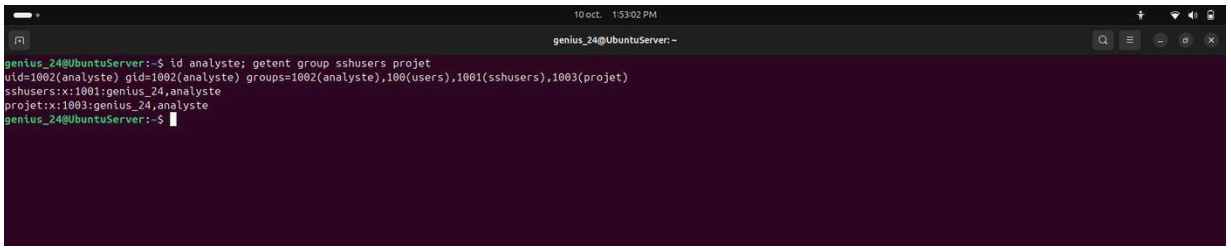


Figure 19 – Compte analyste (sans sudo) et groupes sshusers et projet

Un répertoire partagé /srv/projet a ensuite été créé. Il appartient à root:projet avec les droits 2770 : le groupe projet peut lire et écrire, les autres utilisateurs n'ont aucun accès, et le bit setgid (le « s » dans drwxrws---) fait hériter le groupe aux nouveaux fichiers.

```
sudo mkdir /srv/projet
sudo chown root:projet /srv/projet
sudo chmod 2770 /srv/projet
ls -ld /srv/projet
```

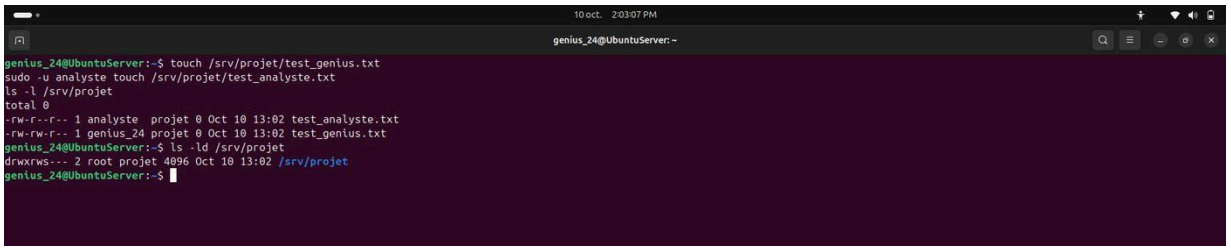


Figure 20 – Répertoire partagé /srv/projet : droits 2770 et fichiers hérités du groupe projet

Le compte analyste a enfin été testé en SSH : la connexion fonctionne et sudo -l confirme l'absence de droits d'administration.

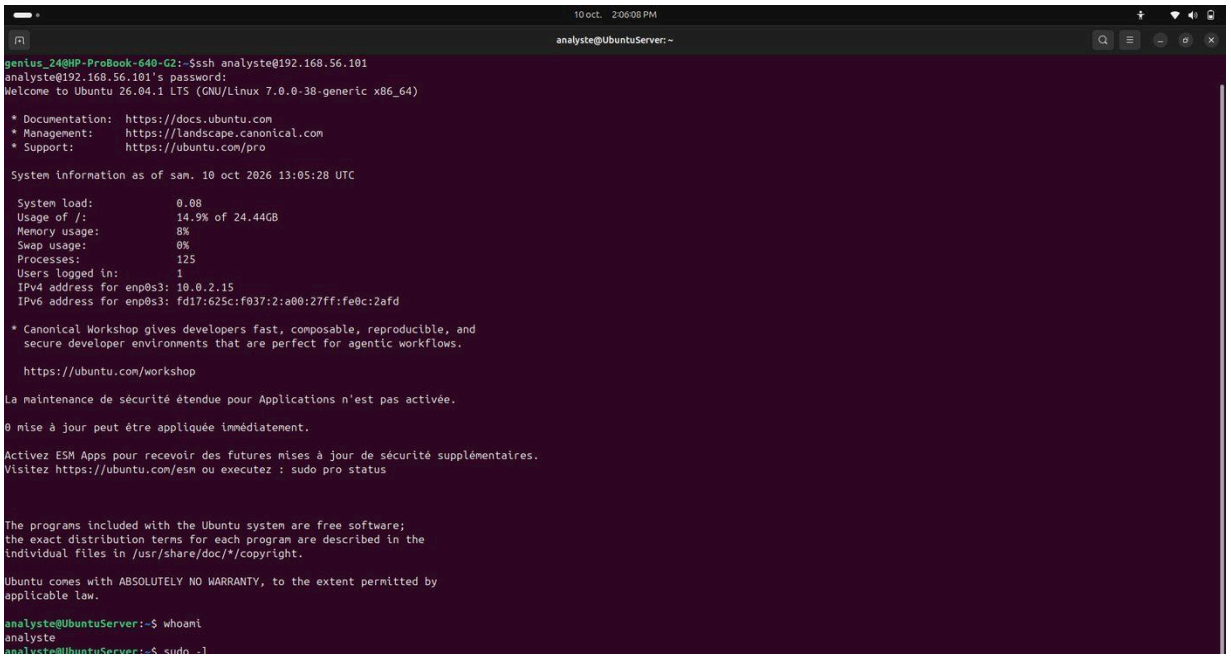


Figure 21 – Connexion SSH d'analyste ; « may not run sudo »

4.2 Pare-feu UFW

UFW a été configuré avant d'être activé, afin de ne pas perdre l'accès d'administration : politique entrante en refus, politique sortante en autorisation, et SSH autorisé uniquement depuis le réseau Host-Only (192.168.56.0/24).

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp
sudo ufw show added
```

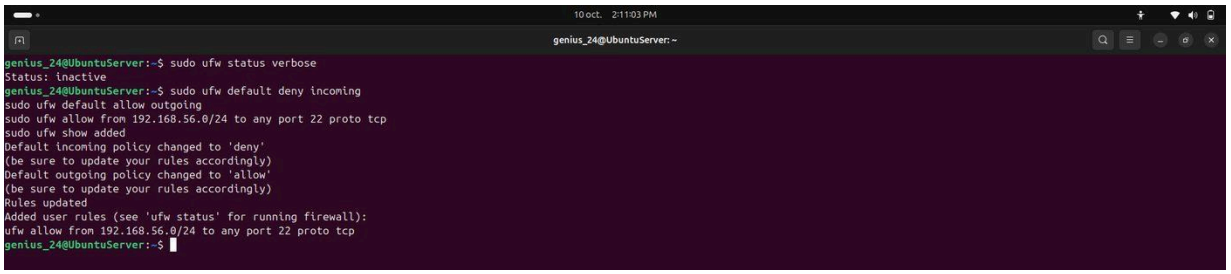


Figure 22 – Règles UFW préparées avant activation

Le pare-feu a ensuite été activé, avec la console VirtualBox gardée ouverte comme accès de secours.

```
sudo ufw enable
sudo ufw status verbose
sudo ufw status numbered
```

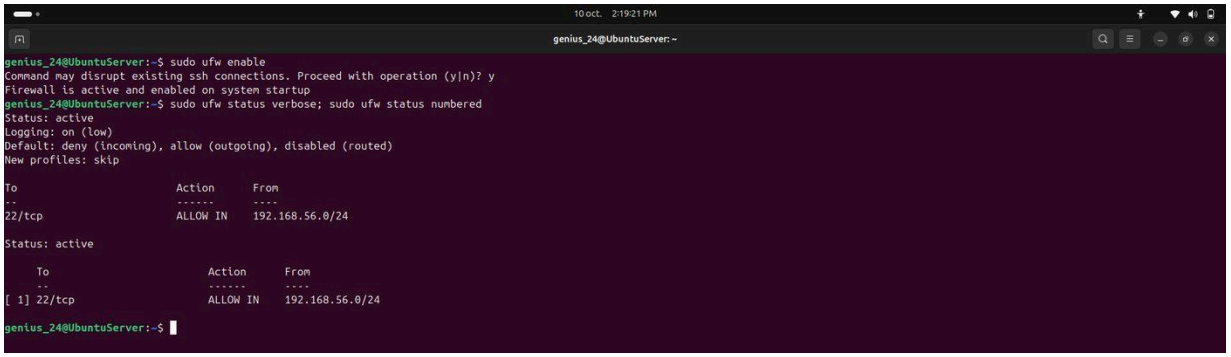


Figure 23 – UFW actif : refus par défaut, SSH autorisé depuis 192.168.56.0/24

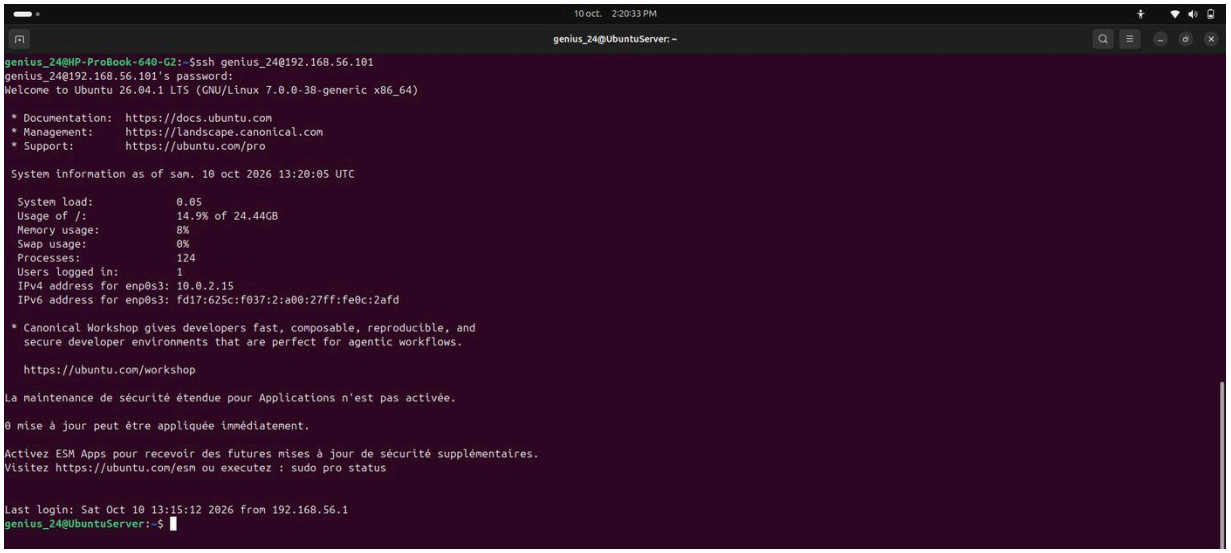


Figure 24 – Nouvelle connexion SSH réussie après l'activation du pare-feu

4.3 Contrôle des ports exposés

La commande ss -lntup sur le serveur montre que seul SSH (sshd, port 22) écoute sur une adresse externe. Les autres services (systemd-resolved, chronyd, client DHCP) sont limités à l'interface locale ou au client DHCP.

```
sudo ss -lntup
```

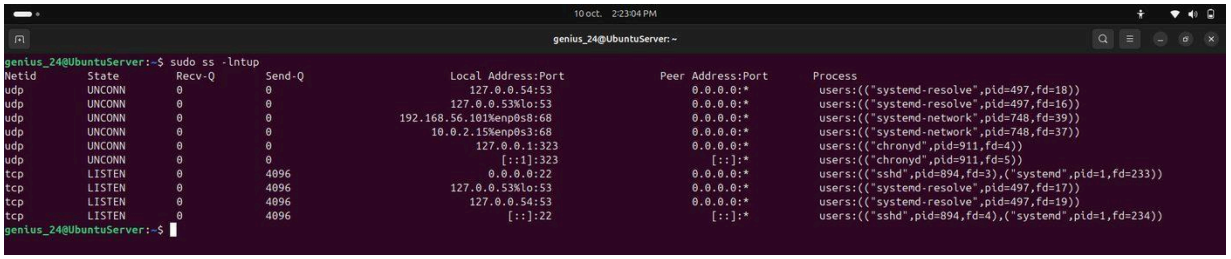


Figure 25 – Ports en écoute : seul SSH est exposé

Depuis l'hôte, un scan Nmap des ports 1 à 1024 indique 1 023 ports filtrés et uniquement 22/tcp ouvert. Un test de connexion sur le port 80 expire (timed out), ce qui confirme le blocage par UFW.

```
nmap -Pn -p 1-1024 192.168.56.101
nc -zv -w 3 192.168.56.101 80
```

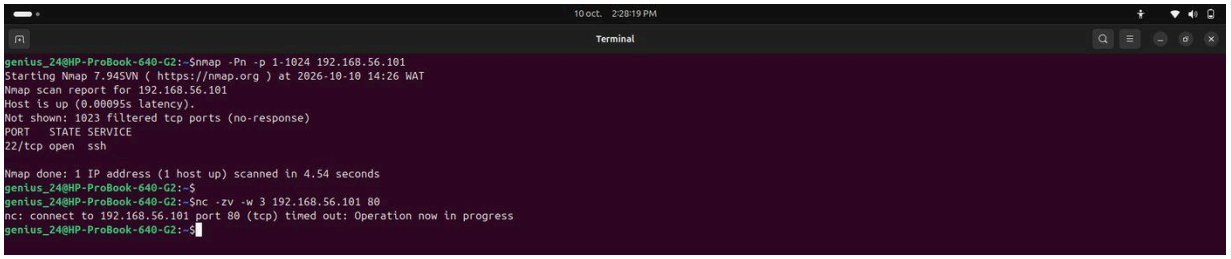


Figure 26 – Scan depuis l'hôte : seul le port 22 est accessible

4.4 Authentification par clés SSH

Avant de désactiver les mots de passe, une paire de clés Ed25519 a été générée sur l'hôte et la clé publique a été déployée sur le serveur pour les deux comptes. Les connexions suivantes s'ouvrent sans mot de passe de compte.

```
ssh-keygen -t ed25519 -C "genius_24@lab-ubuntu"
ssh-copy-id genius_24@192.168.56.101
ssh-copy-id analyste@192.168.56.101
```

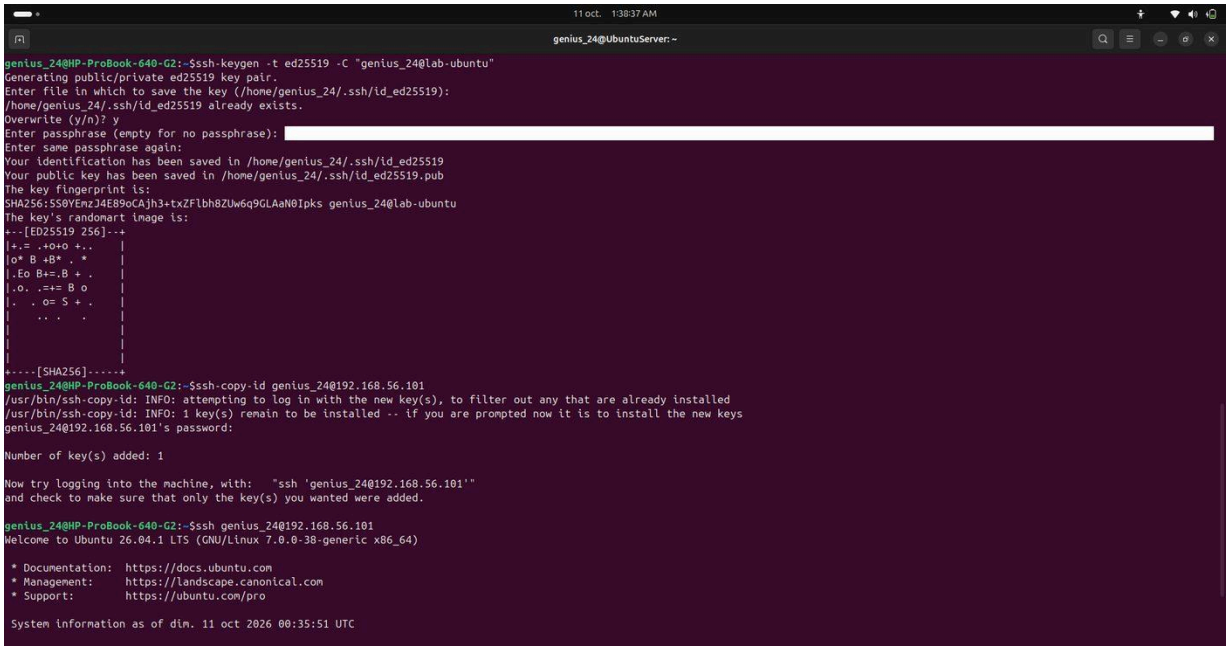


Figure 27 – Génération de la clé Ed25519 et déploiement pour genius_24

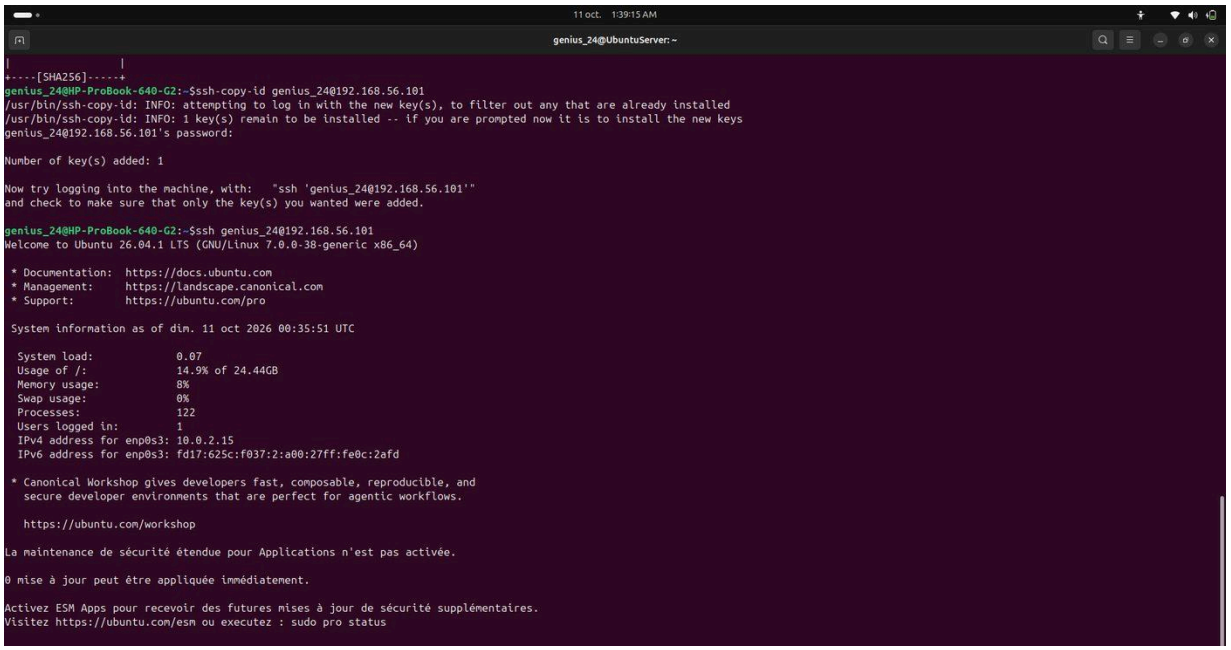


Figure 28 – Connexion par clé réussie

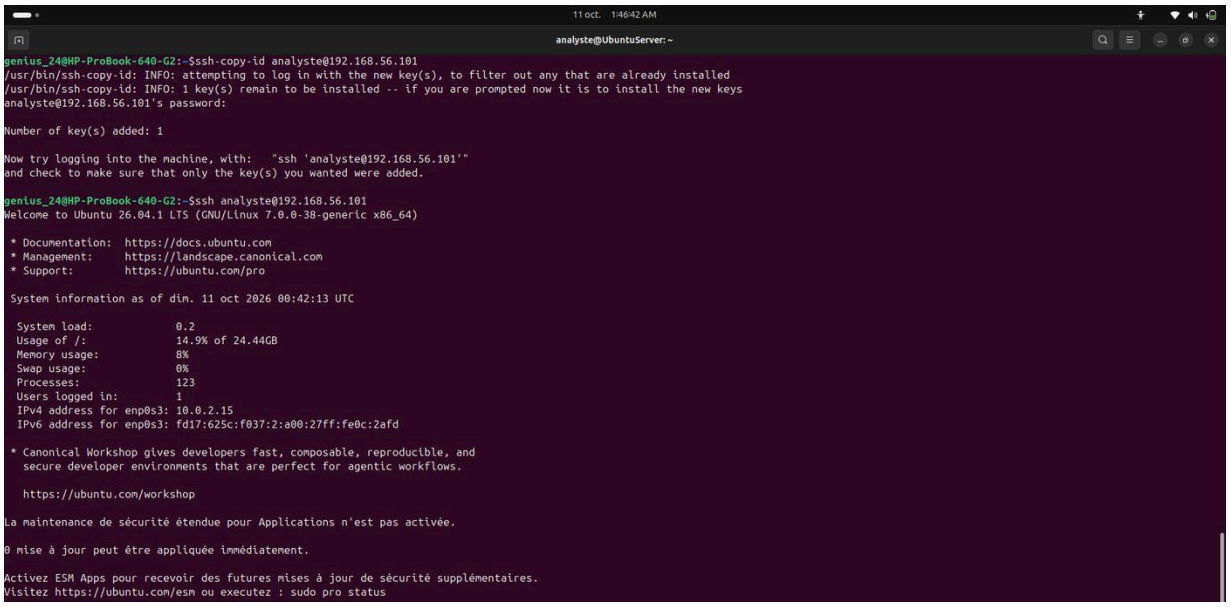


Figure 29 – Déploiement de la clé et connexion par clé du compte analyste

Les journaux du serveur confirment que la clé a bien été utilisée : on y lit des lignes « Accepted publickey » pour genius_24 et analyste, alors que les premières connexions (avant le déploiement des clés) apparaissent en « Accepted password ».

```
sudo journalctl -u ssh --no-pager | grep -i "accepted" | tail -5
```

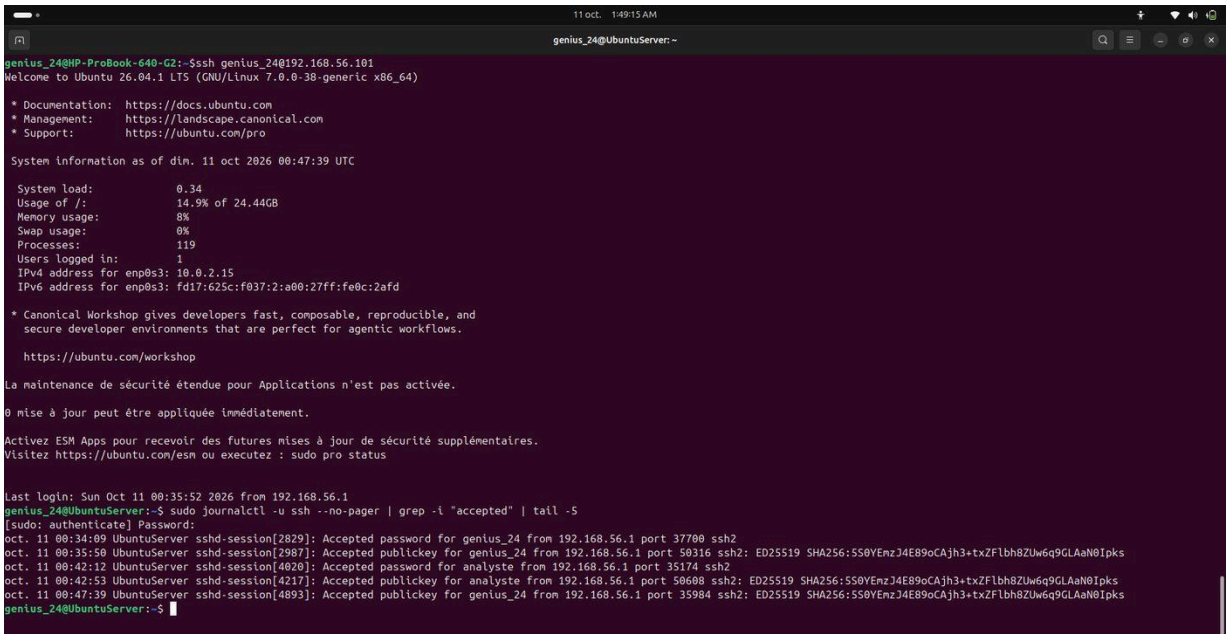


Figure 30 – Journaux SSH : « Accepted publickey » pour les deux comptes

4.5 Durcissement de SSH

Le durcissement a été réalisé dans un fichier dédié, /etc/ssh/sshd_config.d/99-hardening.conf, afin de ne pas modifier le fichier principal. La syntaxe est contrôlée avec sshd -t avant tout rechargement, et la session d'administration est conservée ouverte pendant les tests.

```
# /etc/ssh/sshd_config.d/99-hardening.conf
PermitRootLogin no
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowGroups sshusers
PasswordAuthentication no
KbdInteractiveAuthentication no
```

```
sudo sshd -t
sudo sshd -T | grep -E
'^(permitrootlogin|maxauthtries|logingracetime|x11forwarding|allowgroups)'
sudo systemctl reload ssh
```

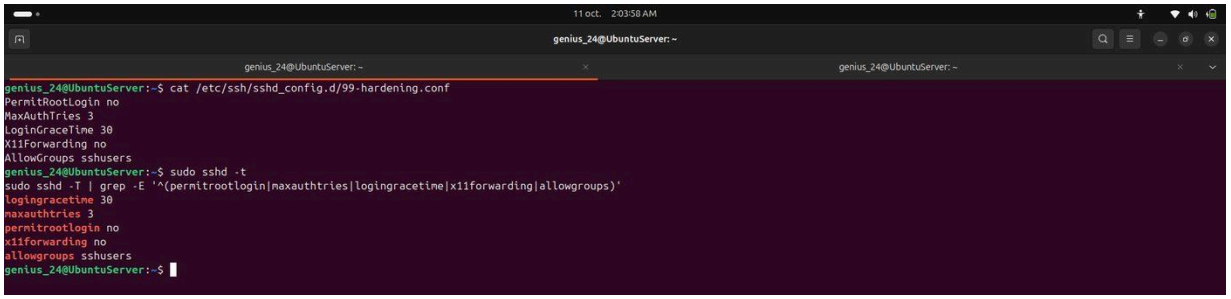


Figure 31 – Première partie du durcissement : valeurs effectives de sshd -T

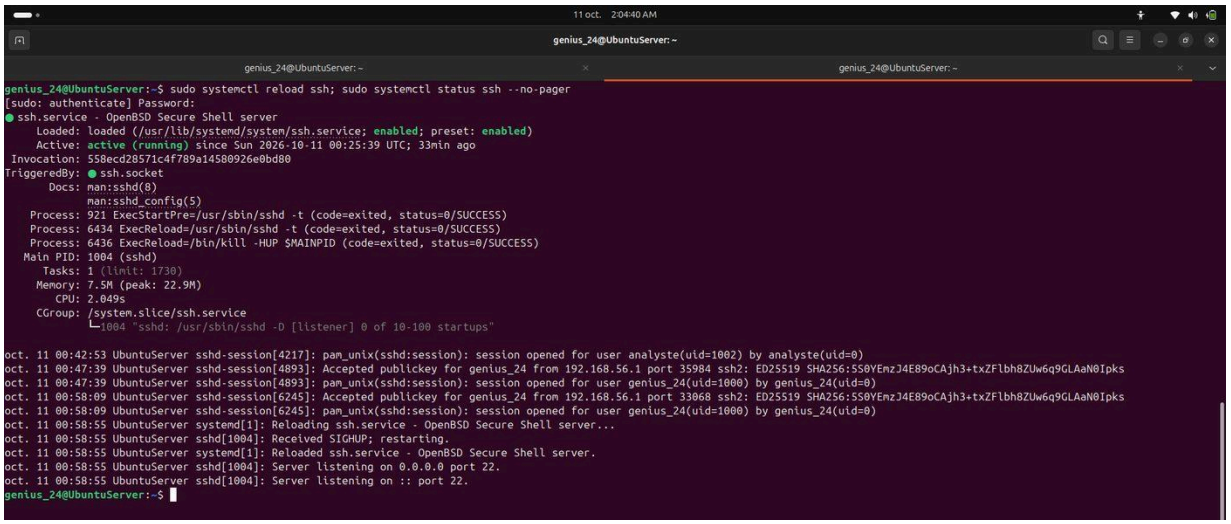


Figure 32 – Rechargement du service SSH (ExecReload en succès)

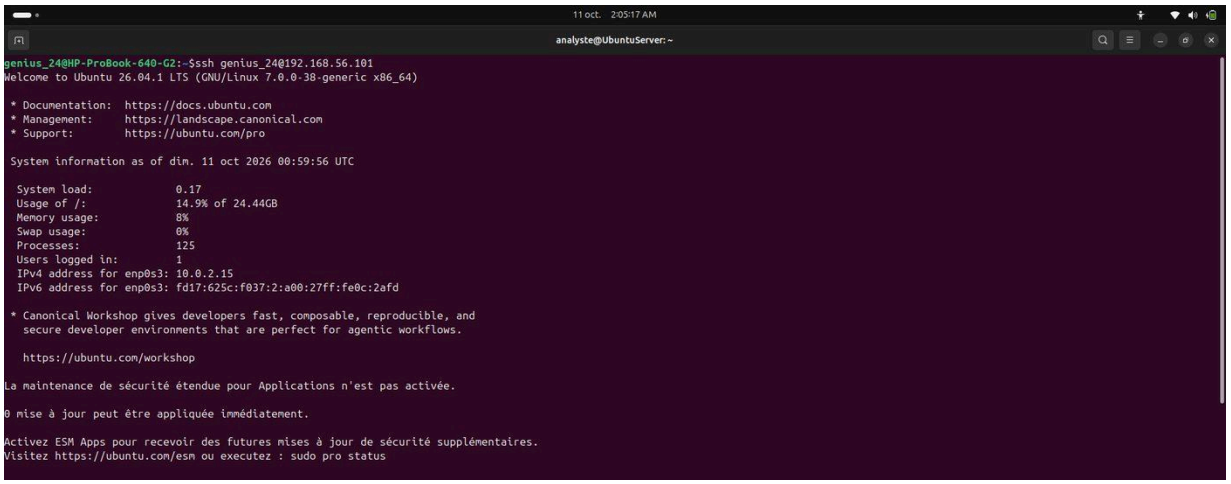


Figure 33 – Connexion de genius_24 après durcissement

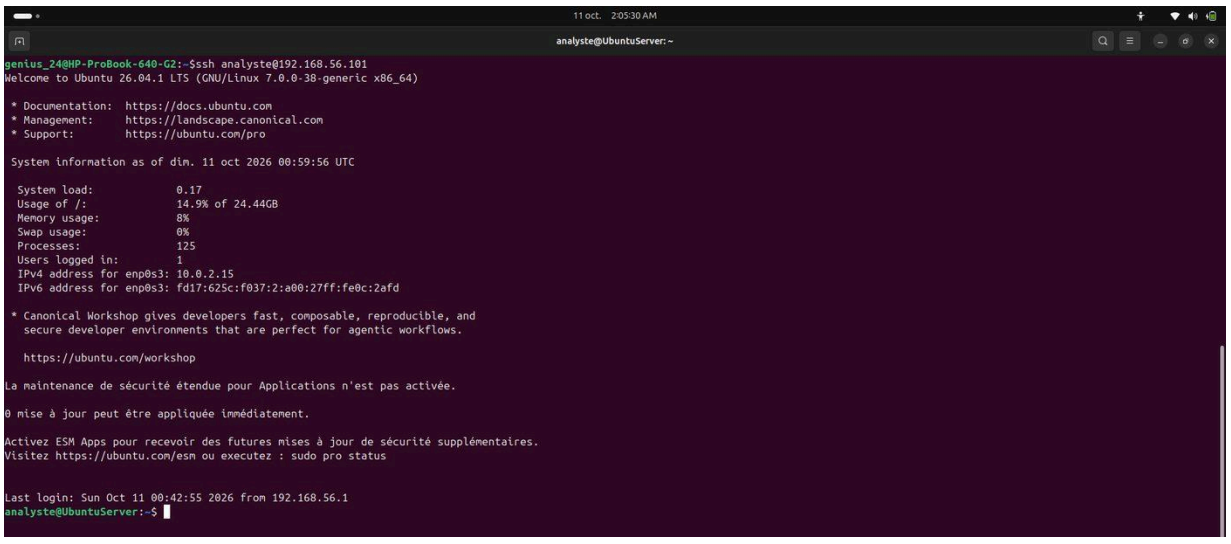


Figure 34 – Connexion d'analyste après durcissement

Dans un second temps, une fois les clés validées pour les deux comptes, l'authentification par mot de passe a été désactivée. La configuration a été vérifiée avec sshd -T avant le rechargement.

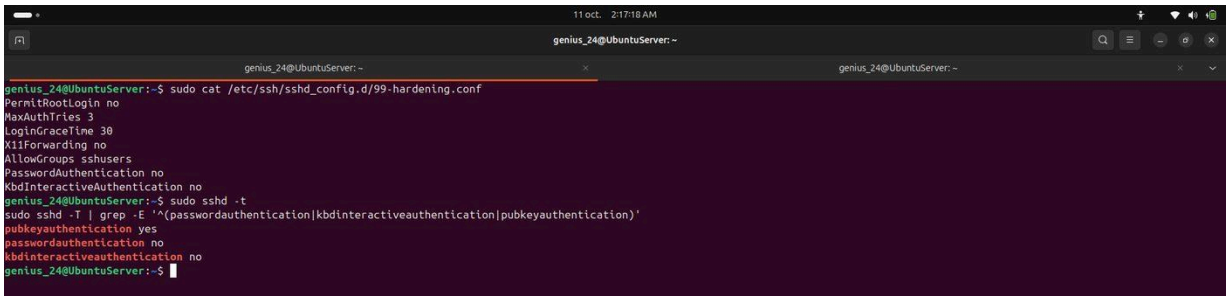


Figure 35 – Mots de passe désactivés : valeurs effectives de sshd -T

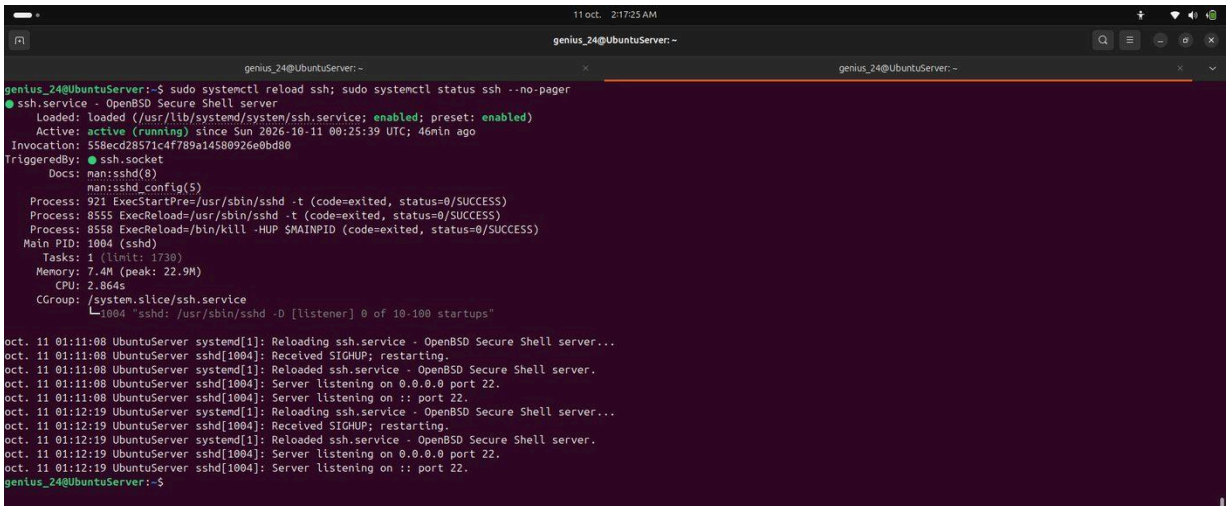


Figure 36 – Rechargement du service après désactivation des mots de passe

Un test sans clé confirme le résultat : la connexion est refusée avec « Permission denied (publickey) », sans qu'aucun mot de passe ne soit demandé.

```
ssh -o PubkeyAuthentication=no genius_24@192.168.56.101
```

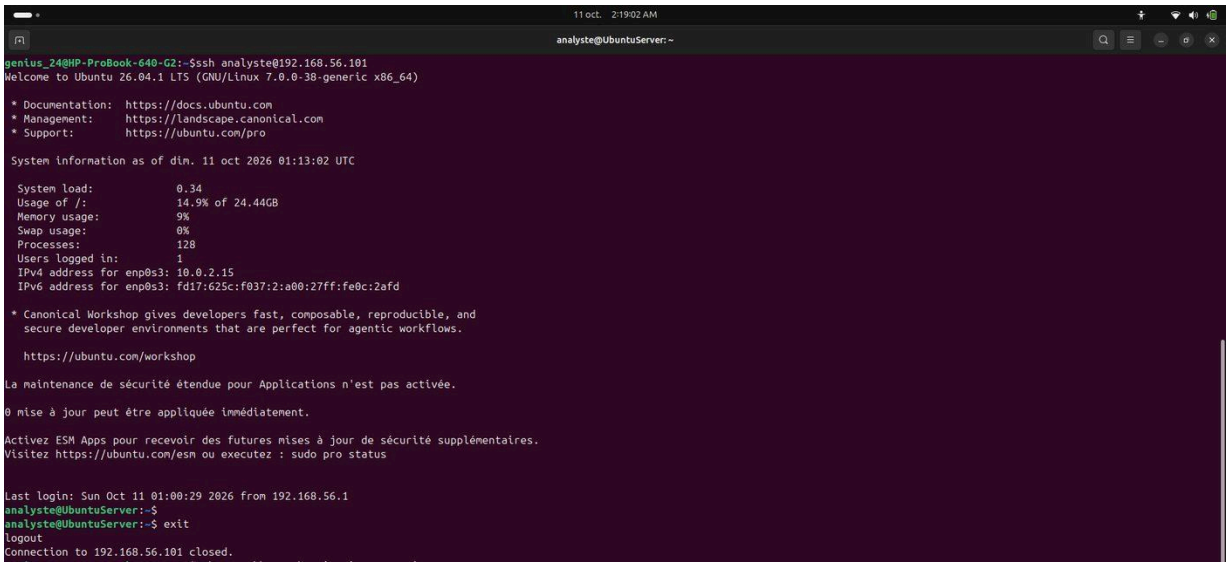


Figure 37 – Connexions par clé réussies et refus de toute connexion sans clé

4.6 Protection contre les tentatives répétées : Fail2ban

Fail2ban 1.1.0 a été installé. Une configuration locale, séparée des fichiers par défaut, active la protection de SSH : bannissement de 15 minutes après 3 échecs en 10 minutes. L'hôte est placé dans la liste d'exceptions (ignoreip) pour éviter de se bloquer pendant les tests.

```
sudo apt install fail2ban
# /etc/fail2ban/jail.d/sshd-local.conf
[sshd]
enabled = true
backend = systemd
maxretry = 3
findtime = 10m
bantime = 15m
ignoreip = 127.0.0.1/8 ::1 192.168.56.1
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
```

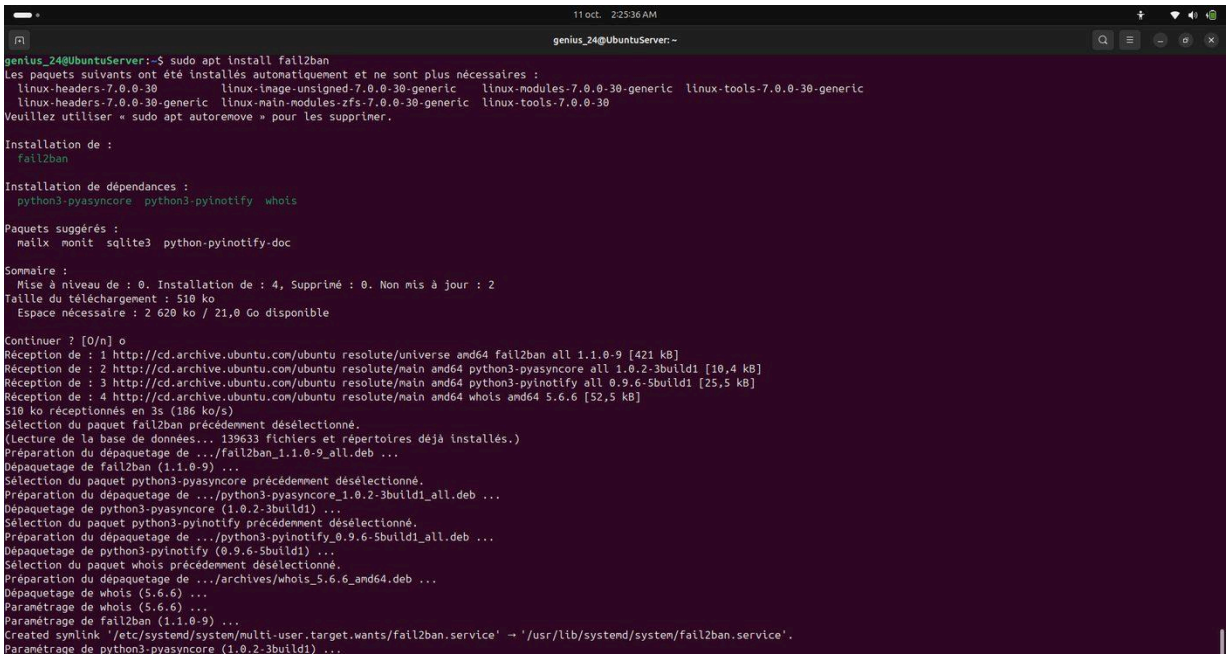


Figure 38 – Installation de Fail2ban

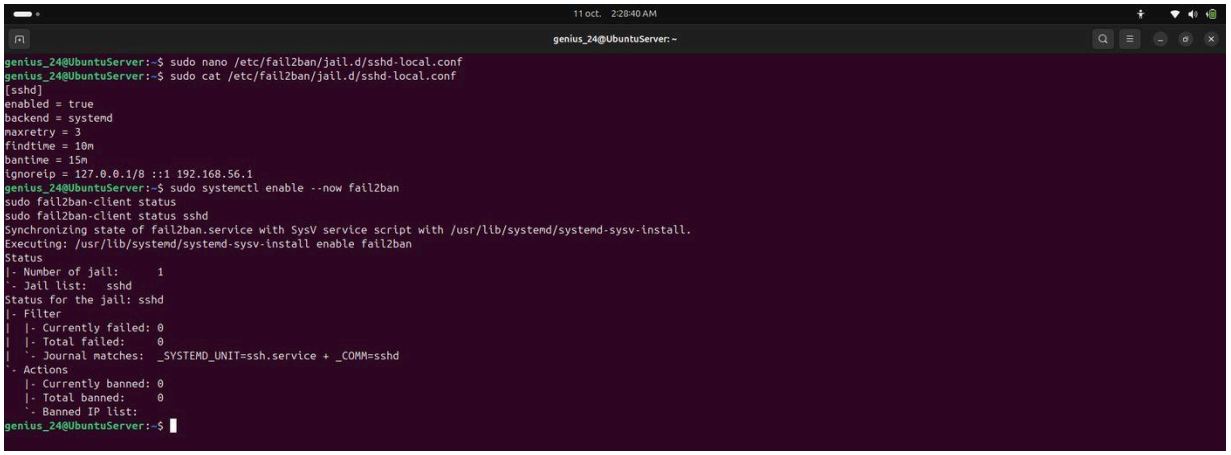


Figure 39 – Configuration de la jail sshd et état initial (0 bannissement)

Test de bannissement : l'adresse de l'hôte a été retirée temporairement de ignoreip, puis trois tentatives avec un utilisateur inexistant ont été effectuées depuis l'hôte. L'adresse 192.168.56.1 a été bannie : la connexion suivante, y compris celle du compte légitime, a été refusée.

```
ssh -o PubkeyAuthentication=no faux_utilisateur@192.168.56.101
sudo fail2ban-client status sshd
```

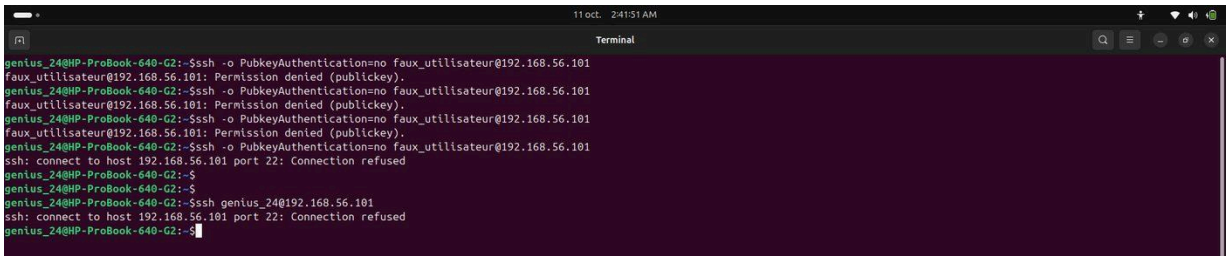


Figure 40 – Échecs puis refus de connexion depuis l'hôte

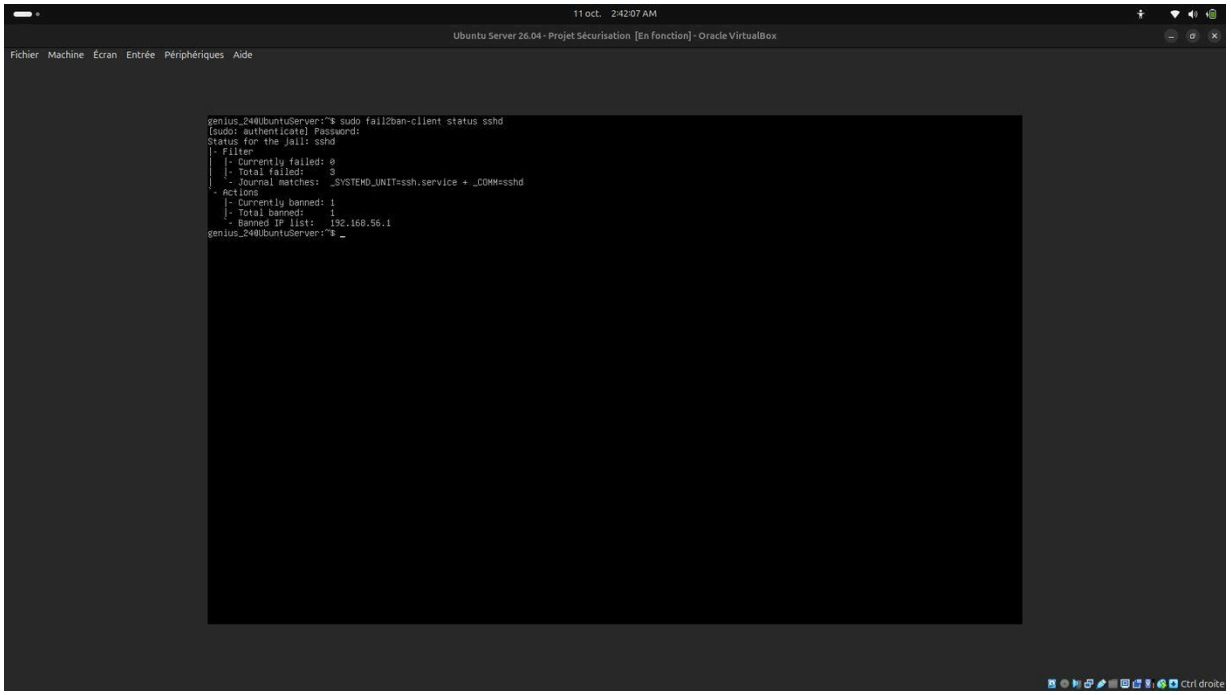


Figure 41 – Console VirtualBox : 1 adresse bannie (192.168.56.1)

L'hôte a ensuite été débanni manuellement, puis remis dans la liste d'exceptions, et l'accès SSH a été rétabli.

```
sudo fail2ban-client set sshd unbanip 192.168.56.1
sudo systemctl restart fail2ban
sudo fail2ban-client status sshd
```

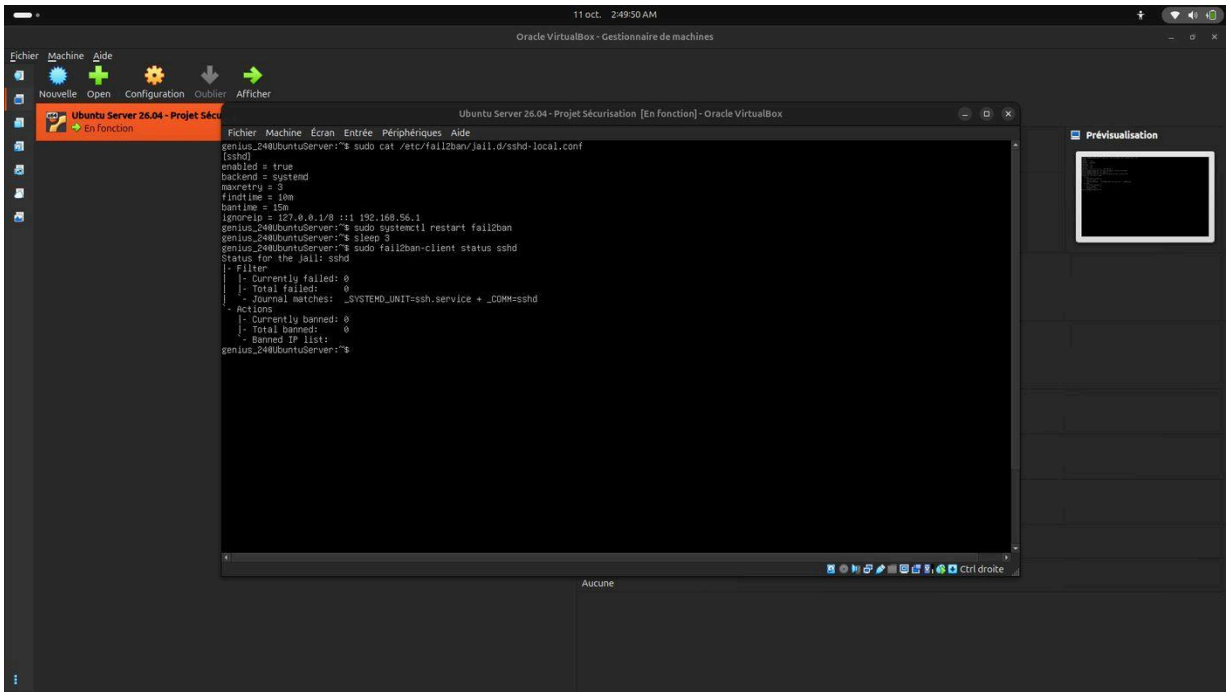


Figure 42 – Débannissement et rétablissement de ignoreip

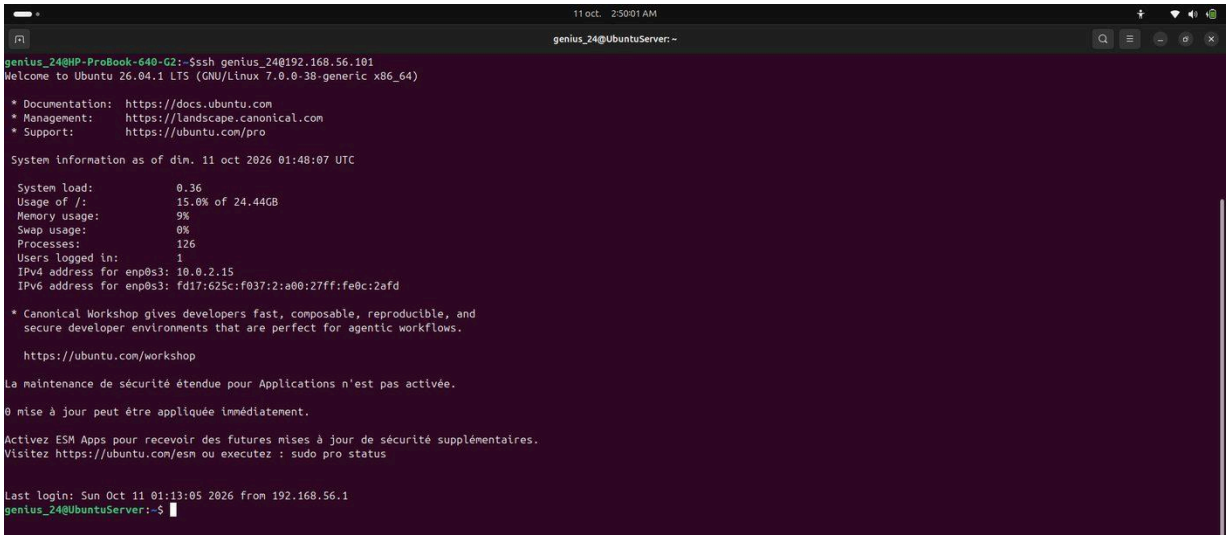


Figure 43 – Connexion SSH rétablie depuis l'hôte

5. Journaux, surveillance et sauvegarde

Sécuriser un serveur ne suffit pas : il faut aussi pouvoir observer ce qui s'y passe et le remettre en état après un incident. Ce chapitre montre comment retrouver la trace des événements dans les journaux, comment contrôler l'utilisation des ressources, et comment sauvegarder puis restaurer les données et les configurations importantes.

5.1 Analyse des journaux

Le journal SSH retrace l'historique des connexions : connexions par mot de passe avant le déploiement des clés, connexions par clé ensuite, et trois lignes « Invalid user faux_utilisateur » correspondant au test de bannissement.

```
sudo journalctl -u ssh --no-pager | grep -E "Invalid user|Failed|Accepted" | tail -15
```

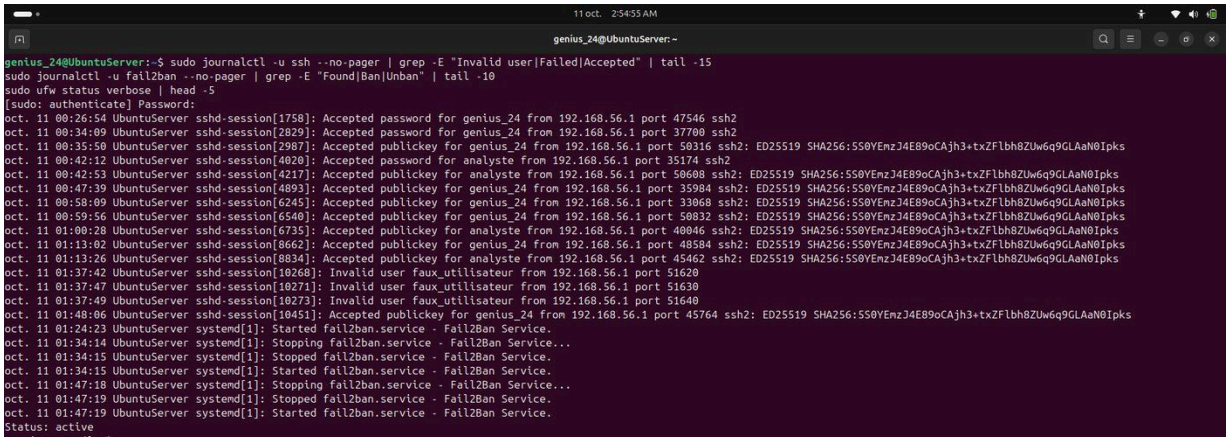


Figure 44 – Journal SSH : connexions par clé et tentatives d'utilisateur invalide

Le fichier /var/log/fail2ban.log confirme toute la chaîne : trois détections (« Found ») à 01:37:42, 01:37:47 et 01:37:49, le bannissement (« Ban ») à 01:37:50, puis le débannissement manuel (« Unban ») à 01:44:43.

```
sudo grep -E "Found|Ban |Unban " /var/log/fail2ban.log | tail -10
```

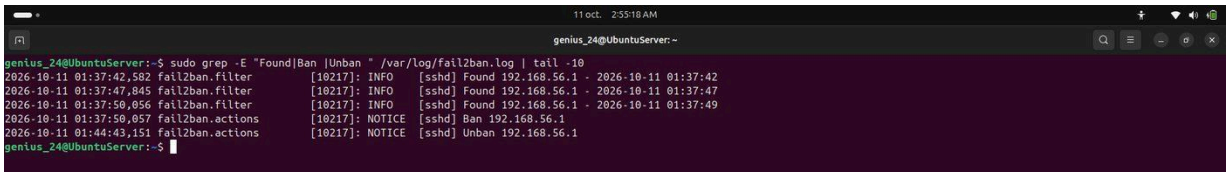


Figure 45 – Journal Fail2ban : détection, bannissement et débannissement

5.2 Surveillance des ressources

Le serveur est au repos après durcissement : charge moyenne de 0,00 / 0,04 / 0,12, environ 493 Mo de mémoire utilisés sur 3,3 Go, swap inutilisé, et 16 % du disque occupé (3,7 Go sur 25 Go). Aucun processus inattendu n'est présent ; fail2ban-server apparaît bien parmi les processus actifs. Un suivi interactif a également été réalisé avec htop (31 tâches, charge très faible).

```
uptime
free -h
df -h /
ps aux --sort=-%mem | head -6
sudo apt install htop
htop
```

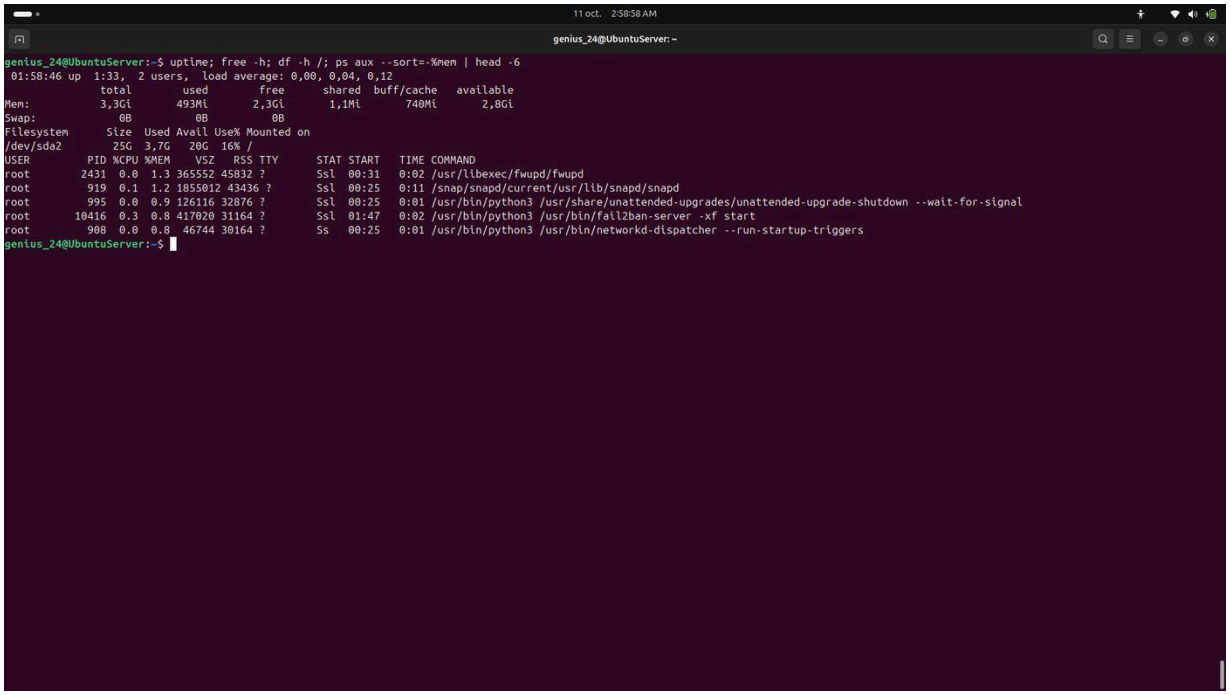


Figure 46 – Charge, mémoire et disque du serveur au repos

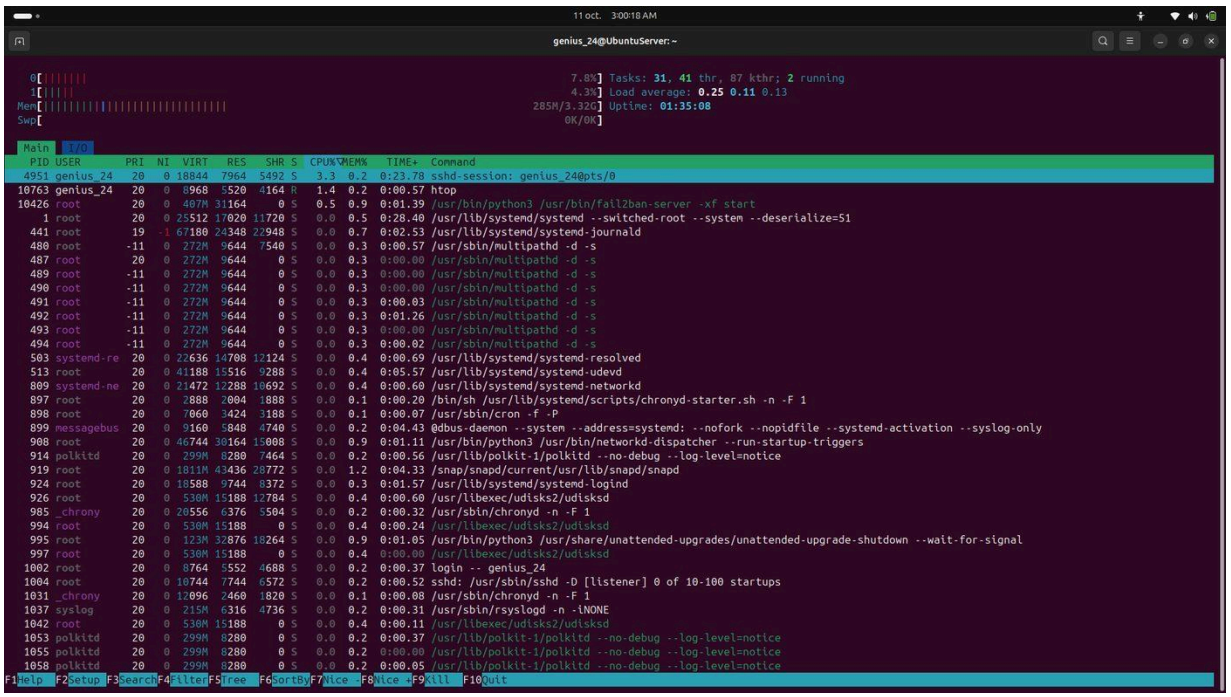


Figure 47 – Suivi interactif des processus avec htop

5.3 Sauvegarde et restauration

Une archive datée a été créée pour le répertoire partagé /srv/projet et les configurations de sécurité (SSH, Fail2ban, UFW). Elle pèse 4 559 octets.

```
sudo mkdir -p /var/backups/projet
sudo tar -czf /var/backups/projet/sauvegarde-$(date +%F).tar.gz /srv/projet
/etc/ssh/sshd_config.d /etc/fail2ban/jail.d /etc/ufw
ls -lh /var/backups/projet
sudo tar -tzf /var/backups/projet/sauvegarde-$(date +%F).tar.gz | head -20
```

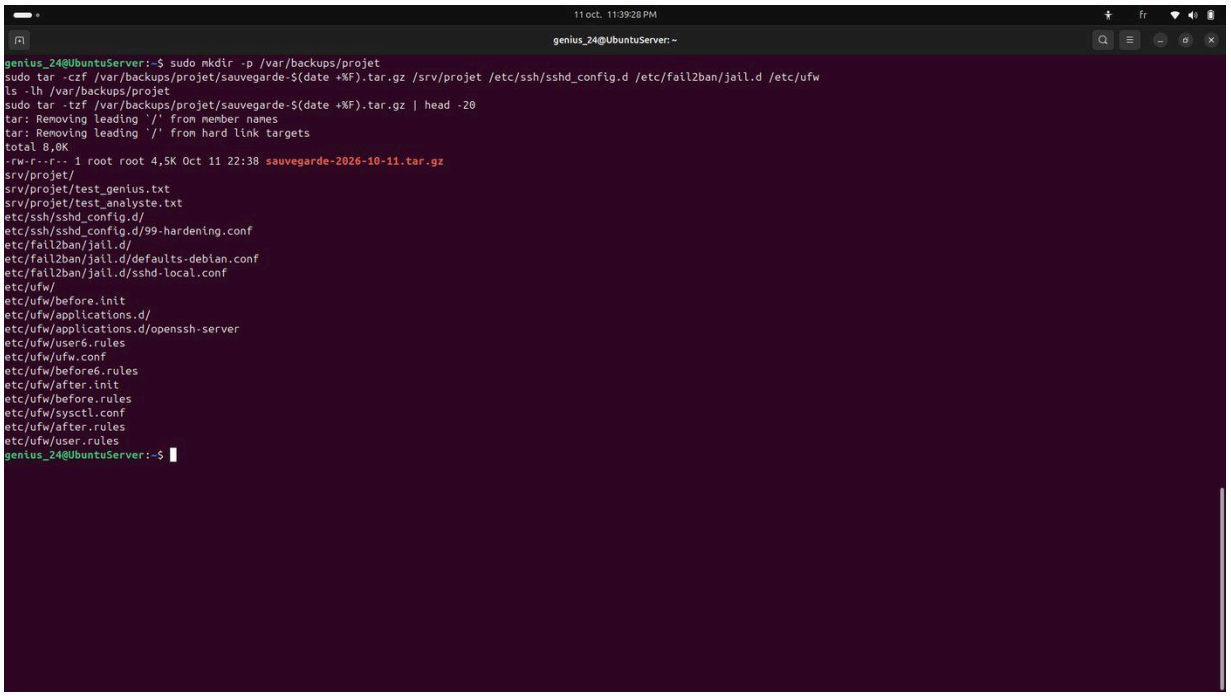


Figure 48 – Création de l'archive datée et contrôle de son contenu

La restauration a été testée dans un dossier temporaire : la comparaison avec les fichiers d'origine ne montre aucune différence pour SSH, Fail2ban et le projet. Un fichier supprimé volontairement (test_genius.txt) a ensuite été récupéré avec son propriétaire, son groupe et sa date d'origine grâce à cp -a. L'archive a enfin été protégée en lecture par root uniquement (droits 600).

```
sudo mkdir /tmp/restauration
sudo tar -xzf /var/backups/projet/sauvegarde-$(date +%F).tar.gz -C /tmp/restauration
sudo diff -r /etc/ssh/sshd_config.d /tmp/restauration/etc/ssh/sshd_config.d && echo "SSH
identique"
sudo rm /srv/projet/test_genius.txt
sudo cp -a /tmp/restauration/srv/projet/test_genius.txt /srv/projet/
sudo rm -r /tmp/restauration
sudo chmod 600 /var/backups/projet/sauvegarde-*.tar.gz
```

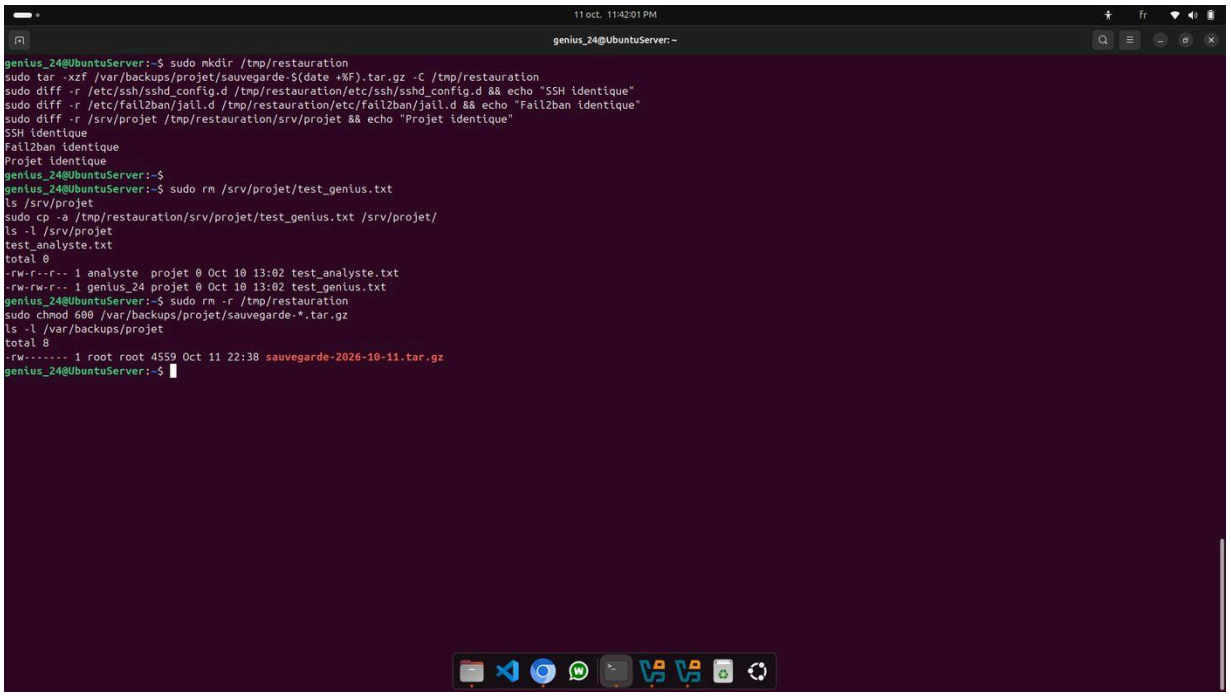


Figure 49 – Restauration vérifiée : contenu identique et fichier récupéré avec ses attributs

Pour se protéger d'une défaillance du disque du serveur, une copie de l'archive a été transférée vers l'hôte avec scp (100 % transférés, 4 559 octets, soit la taille exacte de l'original).

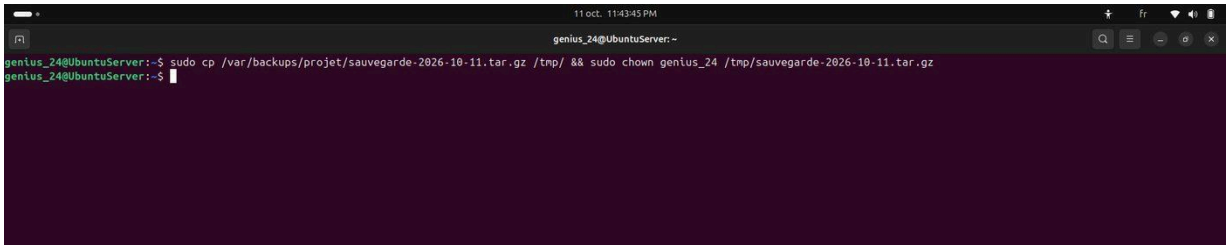


Figure 50 – Préparation de la copie de l'archive sur le serveur

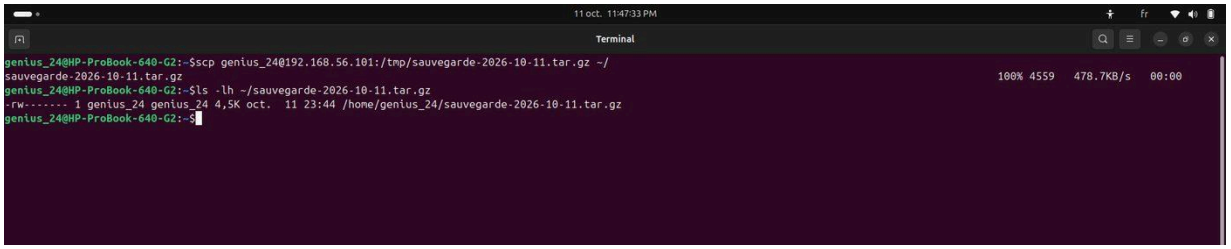


Figure 51 – Transfert de l'archive vers l'hôte avec scp

6. Audit de sécurité avec Lynis

Après la mise en place des mesures, un audit indépendant permet de mesurer le résultat de façon objective plutôt que de se fier à une impression. Lynis analyse la configuration du système et attribue un indice de durcissement, accompagné de recommandations classées par thème.

Lynis 3.1.6 a été installé puis exécuté pour mesurer le niveau de durcissement du serveur.

```
sudo apt install lynis
sudo lynis audit system --quick
sudo grep hardening_index /var/log/lynis-report.dat
```

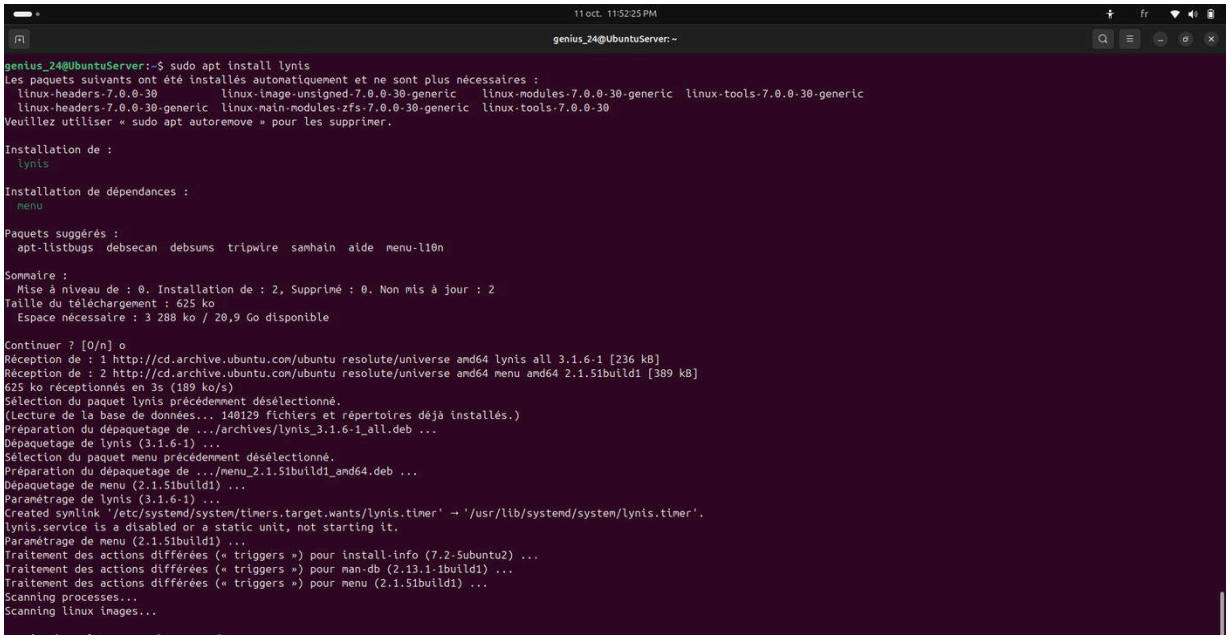


Figure 52 – Installation de Lynis

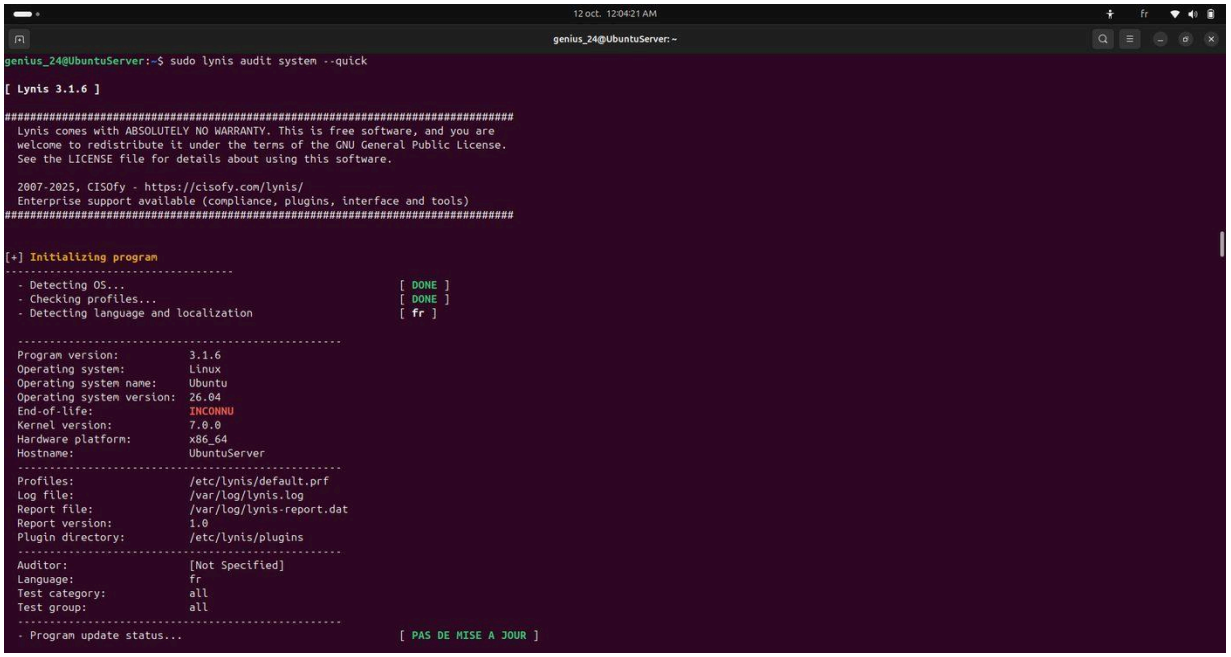


Figure 53 – Début de l'audit Lynis

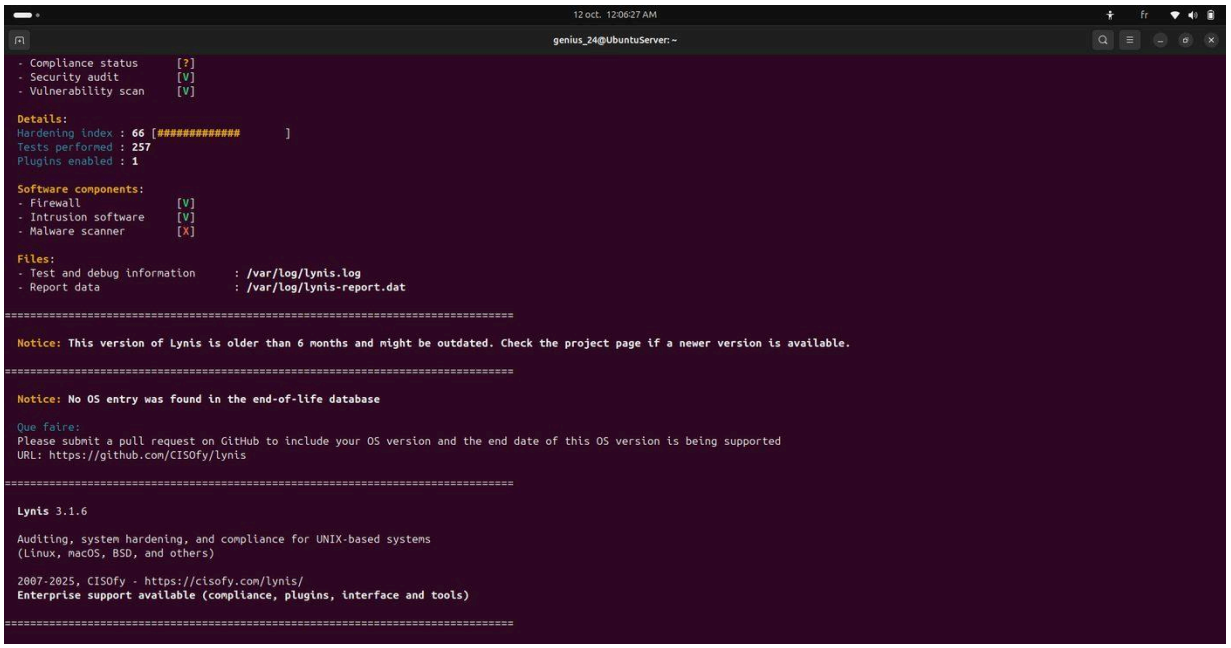


Figure 54 – Résultat : indice de durcissement de 66 (257 tests)

L'indice de durcissement initial est de 66 sur 100, avec 257 tests effectués. Le pare-feu et le logiciel de détection d'intrusion (Fail2ban) sont reconnus ; aucun avertissement (warning) n'est remonté, et seul l'antimalware est signalé comme absent, ce qui est attendu pour ce laboratoire. Deux notices indiquent que la version de Lynis est ancienne et qu'Ubuntu 26.04 n'est pas encore dans sa base de fin de vie ; elles ne concernent pas la configuration du serveur.

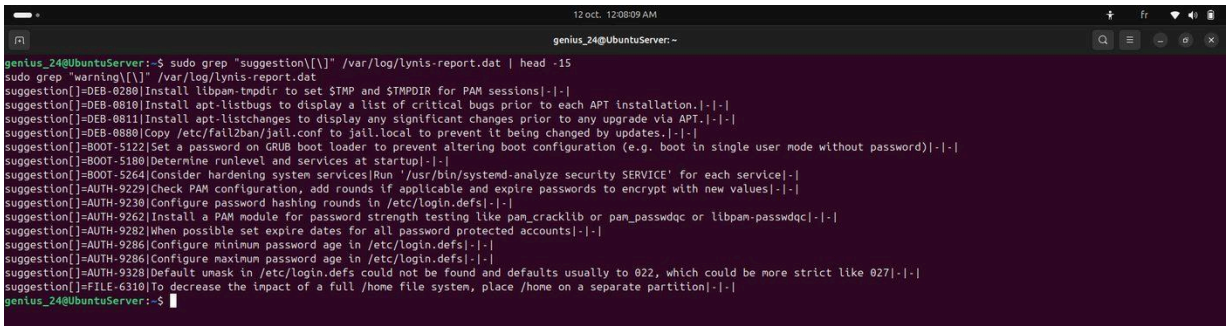


Figure 55 – Principales recommandations Lynis (aucun warning)

Parmi les recommandations, deux mesures simples et sans risque d'accès ont été appliquées : un umask par défaut plus strict (027) dans /etc/login.defs, et des règles d'expiration pour le compte analyste (durée maximale de 90 jours, minimale de 1 jour, avertissement 7 jours avant l'expiration). Le compte administrateur n'a volontairement pas été modifié pour éviter tout risque de verrouillage.

```
sudo sed -i 's/^UMASK.*/UMASK           027/' /etc/login.defs
sudo chage -M 90 -m 1 -W 7 analyste
sudo chage -l analyste
```

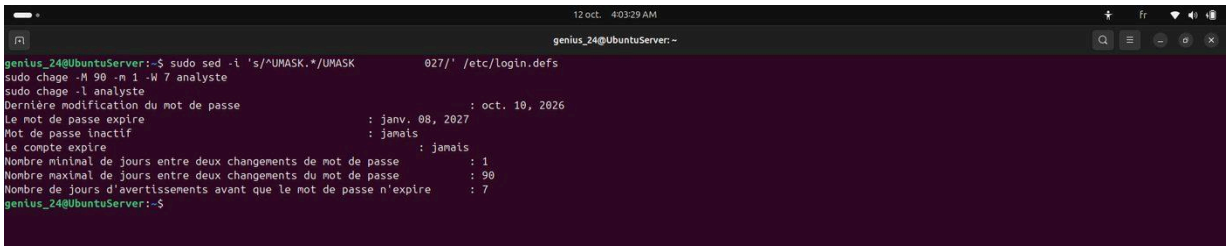


Figure 56 – Umask 027 et règles d'expiration du mot de passe du compte analyste

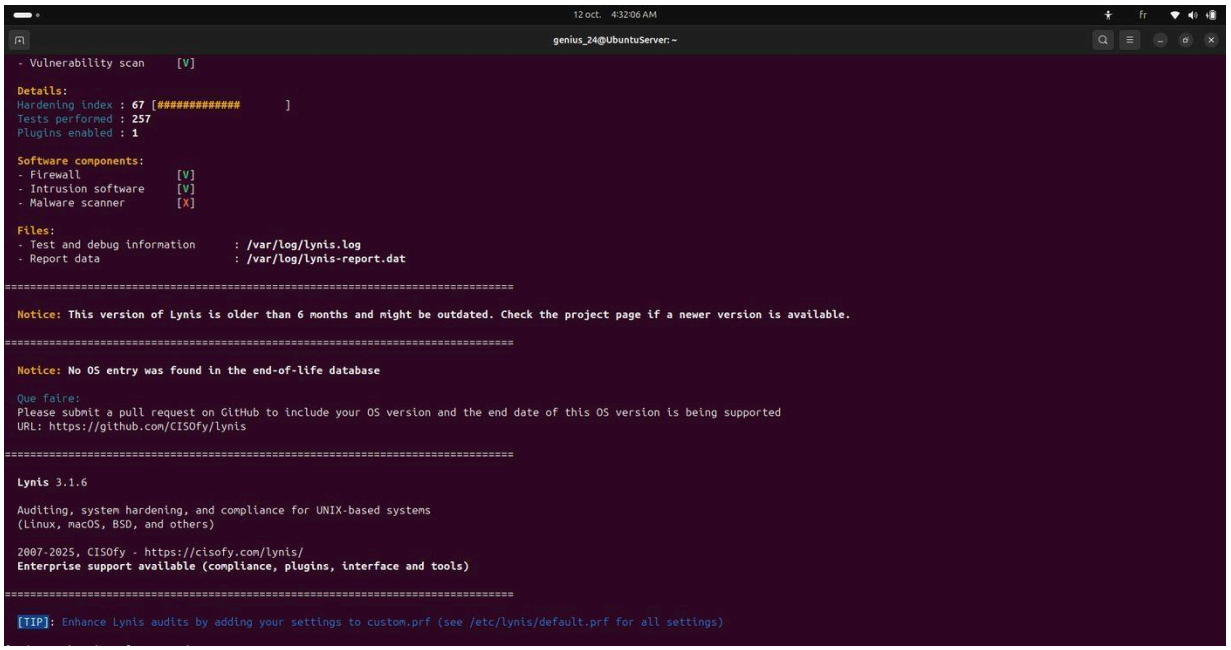


Figure 57 – Nouvel audit : indice de durcissement de 67

Après ces ajustements, l'indice passe de 66 à 67. Les recommandations restantes (mot de passe GRUB, /home sur une partition séparée, module de robustesse des mots de passe, outils APT) ont été volontairement laissées de côté, soit parce qu'elles présentent un risque pour l'accès au serveur, soit parce qu'elles dépassent le cadre de ce laboratoire.

7. Problèmes rencontrés et solutions

Les incidents suivants sont survenus pendant le projet. Chacun a été diagnostiqué à partir du message d'erreur, sans réinstallation ni modification inutile du système.

N°	Problème	Cause	Solution
1	« command not found » à la création du groupe projet	Faute de frappe : groupeadd au lieu de groupadd. Seul le groupe sshusers avait été créé.	Création du groupe avec groupadd projet, puis ajout des comptes aux groupes et vérification avec id et getent group.
2	Refus d'écriture dans /srv/projet pour genius_24	Les groupes d'une session ne sont lus qu'à la connexion : la session ouverte ne connaissait pas encore le groupe projet.	Déconnexion puis reconnexion SSH ; id affiche alors le groupe projet et l'écriture fonctionne.
3	« Connection refused » lors de la première connexion SSH	Adresse saisie incorrecte : 192.162.56.101 au lieu de 192.168.56.101.	Correction de l'adresse. Le service SSH et le serveur n'étaient pas en cause.
4	« Unit ssh.service not found » et journaux sans rapport avec le serveur	Commandes exécutées sur l'hôte (prompt HP-ProBook-640-G2) au lieu du serveur (prompt UbuntuServer).	Reconnexion au serveur avant de relancer les commandes. Le nom d'hôte dans le prompt sert de repère.
5	Échec du rechargement SSH après ajout de la désactivation des mots de passe	Faute de frappe à la ligne 6 : PasswordAuthentification au lieu de PasswordAuthentication.	sshd -t a détecté l'erreur et bloqué le rechargement : la configuration précédente est restée active. Correction de la ligne, nouveau test, puis rechargement.
6	Message « Failed to access socket path … fail2ban.sock »	Le socket n'est pas encore créé juste après le redémarrage du service.	Vérification avec systemctl status fail2ban (active), puis nouvelle commande quelques secondes plus tard : la jail répond.
7	« Network is unreachable » vers 192.168.56.101	L'interface vboxnet0 de l'hôte n'était plus configurée après le redémarrage de l'ordinateur.	Démarrage de la VM dans VirtualBox et rétablissement de l'interface vboxnet0 (192.168.56.1/24) ; la connexion SSH est rétablie.

Enseignement principal : tester la syntaxe (sshd -t), garder une session ouverte pendant tout changement d'accès et disposer d'un accès de secours (console VirtualBox) permet de corriger une erreur sans perdre le contrôle du serveur.

8. Bilan avant / après

Élément	État initial	État final
PermitRootLogin	prohibit-password	no
PasswordAuthentication	yes	no
MaxAuthTries	6	3
X11Forwarding	yes	no
Restriction d'accès SSH	aucune	AllowGroups sshusers
Pare-feu	inactif	UFW actif : refus par défaut, SSH depuis 192.168.56.0/24
Protection anti-intrusion	absente	Fail2ban : bannissement après 3 échecs (15 min)
Ports exposés	–	22/tcp uniquement
Sauvegarde	aucune	Archive datée, restauration testée, copie sur l'hôte
Indice Lynis	66 (audit après durcissement)	67 (après umask et expiration)

8.1 Limites constatées

- La sauvegarde est réalisée manuellement ; elle n'est ni planifiée ni versionnée.
- La copie hors serveur est stockée sur l'hôte, qui est aussi la machine qui héberge la VM.
- La clé privée SSH de l'hôte est indispensable : sa perte ne laisserait que la console VirtualBox comme accès.
- L'adresse de l'hôte figure dans la liste d'exceptions de Fail2ban ; en production, cette exception serait à réexaminer.
- Lynis ne connaît pas encore Ubuntu 26.04 dans sa base de fin de vie ; l'indice est donc à interpréter avec prudence.

8.2 Pistes d'amélioration

- Planifier la sauvegarde avec cron et conserver plusieurs versions.
- Ajouter un mot de passe GRUB et un module de robustesse des mots de passe (libpam-pwquality).
- Copier jail.conf en jail.local, comme le recommande Lynis.
- Mettre en place la supervision des journaux (auditd, logwatch) et les mises à jour de sécurité automatiques.
- Séparer /home sur une partition dédiée lors d'un futur déploiement.

9. Conclusion

9.1 Synthèse

Ce projet a permis de construire un serveur Ubuntu fonctionnel et sécurisé dans un environnement virtualisé, depuis l'installation jusqu'à l'audit final. L'administration à distance repose désormais sur des clés SSH uniquement, avec un accès restreint à un groupe, un pare-feu qui n'expose que le port 22 au réseau d'administration, et une protection automatique contre les tentatives répétées de connexion.

Chaque mesure a été vérifiée par un test concret (connexion refusée sans clé, port 80 bloqué, adresse bannie puis débannie, fichier restauré depuis l'archive), et les erreurs rencontrées ont été documentées avec leur résolution. L'audit Lynis final affiche un indice de durcissement de 67, sans avertissement, et laisse des pistes d'amélioration claires pour la suite.

9.2 Réponse à la problématique

La problématique posée en introduction était de sécuriser un serveur de façon vérifiable, sans perdre l'accès d'administration. Ce résultat a été obtenu grâce à la méthode suivie : valider chaque remplaçant avant de supprimer l'ancien (clés avant mots de passe, règles avant activation du pare-feu), contrôler la syntaxe avant de recharger SSH, et conserver en permanence une session ouverte et une console de secours. Les incidents rencontrés ont tous été résolus sans réinstallation, ce qui illustre l'intérêt de cette prudence.

9.3 Compétences acquises

- Installation, mise à jour et administration d'un serveur Ubuntu en ligne de commande.
- Configuration réseau d'une machine virtuelle (NAT et Host-Only) et diagnostic de connectivité.
- Gestion des utilisateurs, des groupes, des permissions et du bit setgid.
- Configuration d'un pare-feu UFW et vérification par scan de ports (Nmap).
- Mise en place de l'authentification par clés SSH et durcissement de sshd.
- Configuration de Fail2ban et analyse des journaux (journalctl, fichiers de logs).
- Sauvegarde, restauration et copie hors serveur ; audit de sécurité avec Lynis.
- Rédaction d'une documentation technique avec preuves.

9.4 Perspectives

Ce laboratoire constitue une base réutilisable. Les prochaines étapes naturelles sont l'automatisation des sauvegardes, la supervision centralisée des journaux, l'ajout d'un second serveur pour tester des scénarios d'attaque et de défense, et la mise en application des recommandations Lynis restantes. Le travail réalisé ici fournit un socle méthodologique et technique pour ces évolutions.